# CHAPTER 1

# Introduction

# INTRODUCTION

## 1.1 INTRODUCTION TO PROJECT

The **Library Management System (LMS)** project aims to provide an efficient and computerized solution for managing the daily activities of a library. The system is designed to automate traditional library operations such as book management, member registration, book issue and return, fine calculation, and report generation.

The Library Management System allows users to search, issue, and return books easily, while administrators can manage book inventories, user records, transactions, and system reports efficiently. By replacing manual processes, the system reduces paperwork, saves time, and minimizes human errors.

This project provides a centralized database that ensures accurate record keeping, quick access to information, and smooth library operations for both administrators and users.

## 1.2 OBJECTIVE

The main objective of the Library Management System is to develop a user-friendly and reliable system that helps libraries manage their resources effectively. The objectives include:

- To maintain complete information about books, members, and transactions.
- To automate book issue and return processes.
- To reduce manual work and improve accuracy.
- To provide quick search and retrieval of library data.
- To generate reports for administrative decision-making.

## 1.3 PURPOSE & MOTIVATION

**PURPOSE :-** The purpose of the Library Management System is to create a comprehensive application that meets the requirements of both library administrators and users. The primary purposes include:

- **Enhanced User Experience:** To provide an easy-to-use interface for searching books, viewing availability, and managing library accounts.
- **Efficient Management:** To help administrators manage book inventory, user records, issue/return operations, and fine details efficiently.

- **Secure Transactions:** To ensure secure handling of user data and library transactions.
- **Data Management:** To maintain a centralized database for storing book details, member information, and transaction history.

**MOTIVATION :-**With the increasing number of books and users, manual library systems become time-consuming and prone to errors. This project is motivated by the need to modernize library operations using technology, improve efficiency, and provide faster services to users.

## 1.4 PROBLEMS IN THE EXISTING SYSTEM

The existing library system is mostly manual or partially computerized, which leads to several limitations.

### Some problems in the existing system are :-

1. The system relies heavily on manual record keeping.
2. Searching for books and member details is time-consuming.
3. There is a high chance of data redundancy and errors.
4. Report generation is difficult and inaccurate.
5. Users cannot easily track their issued books or due dates.
6. Maintaining large volumes of data is inefficient.

## 1.5 SOLUTION OF THE PROBLEMS

The proposed **Library Management System** overcomes the limitations of the existing system by providing an automated and centralized solution.

### The Administrator can:

1. Add, update, and delete book records.
2. Manage member registration and details.
3. Issue and return books efficiently.
4. Calculate fines automatically.
5. Generate reports related to books, users, and transactions.

### The User can:

1. Search for books easily.
2. View issued books and due dates.
3. Access their account details securely.
4. Perform library activities with minimal effort.

# CHAPTER 2

# System Analysis and Design

# SYSTEM ANALYSIS

## 2.1 INTRODUCTION

After analyzing the requirements of the Library Management System, the next step is system analysis. This phase focuses on understanding the existing system, identifying problems, and defining the requirements of the proposed system.

System analysis helps in studying the current library operations such as book issue, return, record maintenance, and report generation. It also helps in identifying limitations of the existing system and determining how the new system can overcome these issues. A proper analysis ensures that the proposed system is efficient, reliable, and meets user requirements.

## 2.2 ANALYSIS MODEL

The Library Management System follows the **classical Waterfall Model** for software development. This model divides the entire development process into sequential phases, where each phase must be completed before moving to the next.

### Phases of the Waterfall Model are:

- Feasibility Study
- Requirement Analysis and Specification
- System Design
- Coding and Unit Testing
- Integration and System Testing
- Maintenance

Each phase has clearly defined objectives, inputs, and outputs, making the development process systematic and easy to manage.

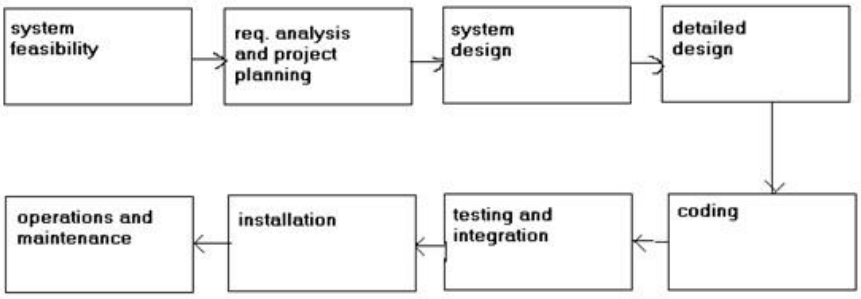



**FIG.1 LIFE CYCLE OF PROJECT IN CLASSICAL WATERFALL MODEL**

### Assumptions of the Waterfall Model:

1. The system requirements are clearly defined and stable.
2. Each phase is completed successfully before the next phase begins.

Each phase of the life cycle has a well-defined starting and ending point (entry and exit criteria) .We now briefly discuss the chief activities carried out during each phase and the corresponding entry and exit criteria

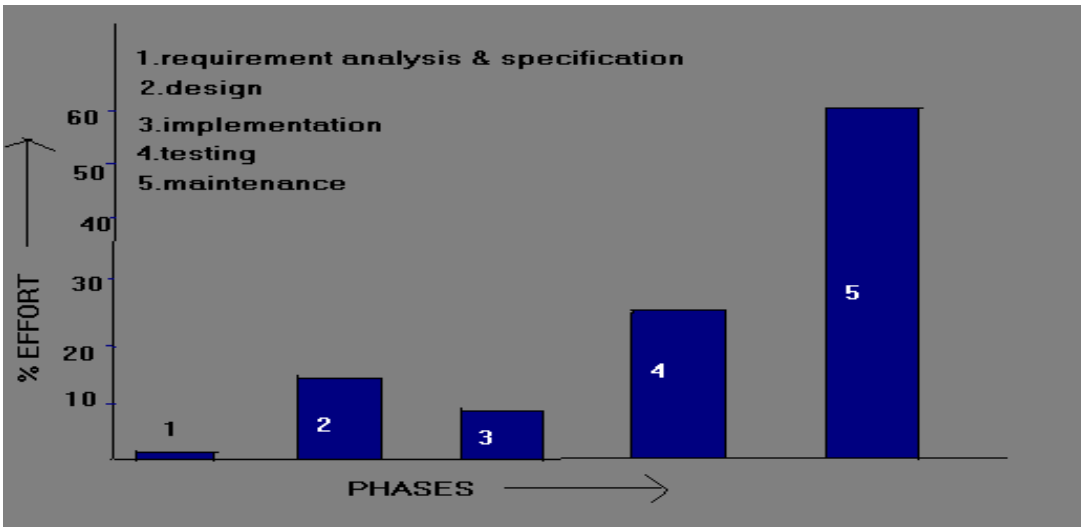



**FIG.2 RELATIVE EFFORT DISTRIBUTION AMONG DIFFERENT OF A TYPICAL PRODUCT**

## 2.3 STUDY OF THE SYSTEM

The Library Management System has been designed with a user-friendly interface that simplifies library operations. The system supports both **Administrator** and **User** interfaces.

### 2.3.1 Administrator Interface

The administrator is responsible for managing the entire library system. The administrator can:

- Add, update, and delete book records
- Manage student/member details
- Issue and return books
- Maintain fine records
- Generate reports related to books and users
- Monitor overall system activity

### 2.3.2 User Interface

The user interface is designed for library members to easily access library services. The user can:

- Search for available books
- View issued books and due dates
- Check fine details
- View personal account information
- Login securely to the system

## 2.4 FEASIBILITY STUDY

The feasibility study determines whether the proposed Library Management System is practical and achievable.

### 2.4.1 Technical Feasibility

The system can be developed using available hardware and software technologies such as VB.NET, MS Access, and Windows OS. No advanced hardware is required.

### 2.4.2 Operational Feasibility

The system is easy to use and does not require extensive training. Library staff and users can operate the system efficiently.

# CHAPTER 3

# Hardware and Software requirement

# HARDWARE AND SOFTWARE REQUIREMENT

## 3.1 HARDWARE AND SOFTWARE SPECIFICATION

### ➢ HARDWARE REQUIREMENTS:

- **Processor** : Intel Pentium IV / Dual Core or above
- **Processor Speed** : 500 MHz or above
- **RAM** : 1 GB minimum
- **Hard Disk** : 200 MB free disk space
- **Monitor** : Standard Color Monitor
- **Keyboard** : Standard Keyboard
- **Mouse** : Optical Mouse

### ➢ SOFTWARE REQUIREMENTS:

- **Operating System** : Windows 10/11
- **Programming Language** : VB.NET
- **Front End Tool** : Visual Studio .NET 2012 Professional Edition
- **Framework** : .NET Framework (Minimum for Deployment)
- **Database / Back End** : MS Access 2007

# CHAPTER 4

# Platform Specification

# Plate form Specification

The Library Management System is developed using a reliable and efficient software platform that supports database connectivity, user-friendly interfaces, and secure data management. The selected platform ensures ease of development, flexibility, and maintainability.

**Operating System : Microsoft Windows 1011**

**Front End : Microsoft Visual Studio .NET 2012**

**Back End : MS Access 2007**

# SELECTED SOFTWARE

## 5.1 MICROSOFT .NET FRAMEWORK

The **.NET Framework** is a software development platform developed by Microsoft that supports the creation and execution of applications such as desktop, web, and database-driven systems. It provides a controlled environment for application development and execution.

The Library Management System uses the .NET Framework because of its robustness, security, and ease of integration with databases like MS Access.

### Features of .NET Framework :

- Supports object-oriented programming.
- Provides language interoperability.
- Automatic memory management.
- Enhanced security and reliability.
- Faster application development.
- Easy deployment and maintenance.

### ➔ Features of the Common Language Runtime

The **Common Language Runtime (CLR)** is the execution engine of the .NET Framework. It manages program execution and provides services such as memory management, exception handling, and security enforcement.

### Functions of CLR :

- Converts source code into Intermediate Language **(IL)**
- Handles garbage collection
- Manages thread execution
- Provides runtime type safety
- Handles exceptions efficiently

The CLR ensures that the Library Management System runs efficiently and securely.

## ➔ .NET Framework Class Library

**Visual Studio .NET** is an integrated development environment (IDE) used to develop .NET-based applications. It provides tools for designing user interfaces, writing code, debugging, and deploying applications.

### Advantages of Visual Studio .NET:

- User-friendly development environment
- Integrated debugging tools
- Supports VB.NET programming
- Easy database connectivity
- Rapid application development

Visual Studio .NET is used as the front-end tool for designing and implementing the Library Management System.

## ➔ Client Application Development

Client applications are the closest to a traditional style of application in Windows-based programming. These are the types of applications that display windows or forms on the desktop, enabling a user to perform a task. Client applications include applications such as word processors and spreadsheets, as well as custom business applications such as data-entry tools, reporting tools, and so on. Client applications usually employ windows, menus, buttons, and other GUI elements, and they likely access local resources such as the file system and peripherals such as printers.

Another kind of client application is the traditional ActiveX control (now replaced by the managed Windows Forms control) deployed over the Internet as a Web page. This application is much like other client applications: it is executed natively, has access to local resources, and includes graphical elements.

In the past, developers created such applications using C/C++ in conjunction with the Microsoft Foundation Classes (MFC) or with a rapid application development (RAD)

environment such as Microsoft® Visual Basic®. The .NET Framework incorporates aspects of these existing products into a single, consistent development environment that drastically simplifies the development of client applications.

The Windows Forms classes contained in the .NET Framework are designed to be used for GUI development. You can easily create command windows, buttons, menus, toolbars, and other screen elements with the flexibility necessary to accommodate shifting business needs.

For example, the .NET Framework provides simple properties to adjust visual attributes associated with forms. In some cases the underlying operating system does not support changing these attributes directly, and in these cases the .NET Framework automatically recreates the forms. This is one of many ways in which the .NET Framework integrates the developer interface, making coding simpler and more consistent.

Unlike ActiveX controls, Windows Forms controls have semi-trusted access to a user's computer. This means that binary or natively executing code can access some of the resources on the user's system (such as GUI elements and limited file access) without being able to access or compromise other resources. Because of code access security, many applications that once needed to be installed on a user's system can now be safely deployed through the Web. Your applications can implement the features of a local application while being deployed like a Web page.

## 5.2 VB.NET

### ➔ ACTIVE X DATA OBJECTS.NET

#### A. ADO.NET OVERVIEW

ADO.NET is an evolution of the ADO data access model that directly addresses user requirements for developing scalable applications. It was designed specifically for the web with scalability, statelessness, and XML in mind.

ADO.NET uses some ADO objects, such as the **Connection** and **Command** objects, and also introduces new objects. Key new ADO.NET objects include the

**Data Set, Data Reader, and Data Adapter.**

The important distinction between this evolved stage of ADO.NET and previous data architectures is that there exists an object -- the DataSet -- that is separate and distinct from any data stores. Because of that, the DataSet functions as a standalone entity. You can think of the DataSet as an always disconnected recordset that knows nothing about the source or destination of the data it contains. Inside a DataSet, much like in a database, there are tables, columns, relationships, constraints, views, and so forth.

A DataAdapter is the object that connects to the database to fill the DataSet. Then, it connects back to the database to update the data there, based on operations performed while the DataSet held the data. In the past, data processing has been primarily connection-based. Now, in an effort to make multi-tiered apps more efficient, data processing is turning to a message-based approach that revolves around chunks of information. At the center of this approach is the DataAdapter, which provides a bridge to retrieve and save data between a DataSet and its source data store. It accomplishes this by means of requests to the appropriate MS-ACCESS commands made against the data store.

The XML-based DataSet object provides a consistent programming model that works with all models of data storage: flat, relational, and hierarchical. It does this by having no 'knowledge' of the source of its data, and by representing the data that it holds as collections and data types. No matter what the source of the data within the DataSet is, it is manipulated through the same set of standard APIs exposed through the DataSet and its subordinate objects.

While the DataSet has no knowledge of the source of its data, the managed provider has detailed and specific information. The role of the managed provider is to connect, fill, and persist the DataSet to and from data stores. The OLE DB and Ms-Access.NET Data Providers (System.Data.OleDb and System.Data.Ms-Access Client) that are part of the .Net Framework provide four basic objects: the Command, Connection, DataReader and DataAdapter. In the remaining sections of this document, we'll walk through each part of the DataSet and the OLE DB/Ms-Access.NET Data Providers explaining what they are, and how to program against them.

The following sections will introduce you to some objects that have evolved, and some that are new. These objects are:

- **Connections. For connection to and managing transactions against a database.**
- **Commands. For issuing MS-ACCESS commands against a database.**
- **Data Readers**. For reading a forward-only stream of data records from a Ms-Access data source.
- **Data Sets**. For storing, removing and programming against flat data, XML data and relational data.
- **Data Adapters**. For pushing data into a **Data Set**, and reconciling data against a database.

When dealing with connections to a database, there are two different options: Ms-Access.NET Data Provider (System.Data.Ms-Access Client) and OLE DB .NET Data Provider (System.Data.OleDb). In these samples we will use the Ms-Access.NET Data Provider. These are written to talk directly to Microsoft MS-ACCESS Server.

The OLE DB .NET Data Provider is used to talk to any OLE DB provider (as it uses OLE DB underneath).

**Connections**

Connections are used to 'talk to' databases, and are represented by provider-specific classes such as **MS-ACCESS Connection**. Commands travel over connections and result sets are returned in the form of streams which can be read by a **Data Reader** object, or pushed into a **Data Set** object.

**Commands**

Commands contain the information that is submitted to a database, and are represented by provider-specific classes such as **MS-ACCESS Command**. A command can be a stored procedure call, an UPDATE statement, or a statement that returns results. You can also use input and output parameters, and return values as part of your command syntax. The example below shows how to issue an INSERT statement against the **North wind** database.

**Data Readers**

The Data Reader object is somewhat synonymous with a read-only/forward-only cursor over data. The Data Reader API supports flat as well as hierarchical data. A Data Reader object is returned after executing a command against a database. The format of the returned Data Reader object is different from a record set. For example, you might use the Data Reader to show the results of a search list in a web page.

**Data Sets and Data Adapters**

**DataSets**

The **Data Set** object is similar to the ADO **Record set** object, but more powerful, and with one other important distinction: the **Data Set** is always disconnected. The **Data Set** object represents a cache of data, with database-like structures such as tables, columns, relationships, and constraints. However, though a **Data Set** can and does behave much like a database, it is important to remember that **Data Set** objects do not interact directly with databases, or other source data. This allows the developer to work with a programming model that is always consistent, regardless of where the source data resides. Data coming from a database, an XML file, from code, or user input can all be placed into **Data Set** objects. Then, as changes are made to the **Data Set** they can be tracked and verified before updating the source data. The **Get Changes** method of the **Data Set** object actually creates a second **Dat Set** that contains only the changes to the data. This **Data Set** is then used by a **Data Adapter** (or other objects) to update the original data source.

The **DataSet** has many XML characteristics, including the ability to produce and consume XML data and XML schemas. XML schemas can be used to describe schemas interchanged via WebServices. In fact, a **DataSet** with a schema can actually be compiled for type safety and statement completion.

### B. DATA ADAPTERS (OLEDB/MS-ACCESS )

The **Data Adapter** object works as a bridge between the **Data Set** and the source data. Using the provider-specific **Ms-Access Data Adapter** (along with its associated **Ms-Access Command** and **Ms-Access Connection**) can increase overall performance when working with a Microsoft Ms-Accessdatabases. For other OLE DB-supported databases, you would use the **OleDbDataAdapter** object and its associated **OleDbCommand** and **OleDbConnection** objects.

The **DataAdapter** object uses commands to update the data source after changes have been made to the **DataSet**. Using the **Fill** method of the **DataAdapter** calls the SELECT command; using the **Update** method calls the INSERT, UPDATE or DELETE command for each changed row. You can explicitly set these commands in order to control the statements used at runtime to resolve changes, including the use of stored procedures. For ad-hoc scenarios, a **CommandBuilder** object can generate these at run-time based upon a select statement. However, this run-time generation requires an extra round-trip to the server in order to gather required metadata, so explicitly providing the INSERT, UPDATE, and DELETE commands at design time will result in better run-time performance.

1. ADO.NET is the next evolution of ADO for the .Net Framework.
2. ADO.NET was created with n-Tier, statelessness and XML in the forefront. Two new objects, the **DataSet** and **DataAdapter**, are provided for these scenarios.
3. ADO.NET can be used to get data from a stream, or to store data in a cache for updates.
4. There is a lot more information about ADO.NET in the documentation.
5. Remember, you can execute a command directly against the database in order to do inserts, updates, and deletes. You don't need to first put data into a **DataSet** in order to insert, update, or delete it.
6. Also, you can use a **DataSet** to bind to the data, move through the data, and navigate data relationships.

## 5.3 <u>Ms –Access</u>

### ➔ DATABASE

A database management, or DBMS, gives the user access to their data and helps them transform the data into information. Such database management systems include

dBase, paradox, IMS, Ms-Access. These systems allow users to create, update and extract information from their database.

A database is a structured collection of data. Data refers to the characteristics of people, things and events. Ms-Access stores each data item in its own fields. the fields relating to a particular person, thing or event are bundled together to form a single complete unit of data, called a record (it can also be referred to as raw or an occurrence). Each record is made up of a number of fields. No two fields in a record can have the same field name.

During an Ms-Access Database design project, the analysis of your business needs identifies all the fields or attributes of interest. If your business needs change over time, you define any additional fields or change the definition of existing fields.

**Ms-Access Tables**

Ms-Access stores records relating to each other in a table. Different tables are created for the various groups of information. Related tables are grouped together to form a database.

**Primary Key**

Every table in Ms-Access has a field or a combination of fields that uniquely identifies each record in the table. The Unique identifier is called the Primary Key, or simply the Key. The primary key provides the means to distinguish one record from all other in a table. It allows the user and the database system to identify, locate and refer to one particular record in the database.

**Relational Database**

Sometimes all the information of interest to a business operation can be stored in one table. Ms-Access makes it very easy to link the data in multiple tables. Matching an employee to the department in which they work is one example. This is what makes Ms-Access a relational database management system, or RDBMS. It stores data in two or more tables and enables you to define relationships between the table and enables you to define relationships between the tables.

**Foreign Key**

When a field is one table matches the primary key of another field is referred to as a foreign key. A foreign key is a field or a group of fields in one table whose values match those of the primary key of another table.

**Referential Integrity**

Not only does Ms-Access allow you to link multiple tables, it also maintains consistency between them. Ensuring that the data among related tables is correctly matched is referred to as maintaining referential integrity.

**Data Abstraction:**

A major purpose of a database system is to provide users with an abstract view of the data. This system hides certain details of how the data is stored and maintained. Data abstraction is divided into three levels.

**Physical level**: This is the lowest level of abstraction at which one describes how the data are actually stored.

**Conceptual Level**: At this level of database abstraction all the attributed and what data are actually stored is described and entries and relationship among them.

**View level**: This is the highest level of abstraction at which one describes only part of the database.

➔ **ADVANTAGE OF RDBMS**

- Redundancy can be avoided
- Inconsistency can be eliminated
- Data can be Shared
- Standards can be enforced
- Security restrictions ca be applied
- Integrity can be maintained
- Conflicting requirements can be balanced
- Data independence can be achieved.

➔ **DISADVANTAGE OF RDBMS**

A significant disadvantage of the DBMS system is cost. In addition to the cost of purchasing of developing the software, the hardware has to be upgraded to allow for the extensive programs and the workspace required for their execution and storage. While centralization reduces duplication, the lack of duplication requires that the database be adequately backed up so that in case of failure the data can be recovered.

➔ **FEATURES OF MS-ACCESS (RDBMS)**

MS-ACCESS is one of the leading database management systems (DBMS) because it is the only Database that meets the uncompromising requirements of today's most demanding information systems. From complex decision support systems (DSS) to the most rigorous online transaction processing (OLTP) application, even application

that require simultaneous DSS and OLTP access to the same critical data, Ms-Access leads the industry in both performance and capability

MS-ACCESS is a truly portable, distributed, and open DBMS that delivers unmatched performance, continuous operation and support for every database.

MS-ACCESSRDBMS is high performance fault tolerant DBMS which is specially designed for online transactions processing and for handling large database application.

MS-ACCESS with transactions processing option offers two features which contribute to very high level of transaction processing throughput.

**Enterprise wide Data Sharing**

The unrivaled portability and connectivity of the MS-ACCESSDBMS enables all the systems in the organization to be linked into a singular, integrated computing resource.

**Portability**

Ms-Access fully portable to more than 80 distinct hardware and operating systems platforms, including UNIX, MSDOS, OS/2, Macintosh and dozens of proprietary platforms. This portability gives complete freedom to choose the database sever platform that meets the system requirements.

**Open Systems**

MS-ACCESS offers a leading implementation of industry –standard MS-ACCESS MS-ACCESS open architecture integrates MS-ACCESS and non –MS-ACCESSDBMS with industry's most comprehensive collection of tools, application, and third party software products MS-ACCESS Open architecture provides transparent access to data from other relational database and even non-relational database.

**Distributed Data Sharing**

MS-ACCESS Server's networking and distributed database capabilities to access data stored on remote server with the same ease as if the information was stored on a single local computer. A single MS-ACCESS statement can access data at multiple sites. You can store data where system requirements such as performance, security or availability dictate.

**Unmatched Performance**

The most advanced architecture in the industry allows the MS-ACCESSDBMS to deliver unmatched performance.

**Sophisticated Concurrency Control**

Real World applications demand access to critical data. With most database Systems application becomes "contention bound" – which performance is limited not by the CPU power or by disk I/O, but user waiting on one another for data access . Ms-Access employs full, unrestricted row-level locking and contention free queries to minimize and in many cases entirely eliminates contention wait times

**No I/O Bottlenecks**

MS-ACCESS Server's fast commit groups commit and deferred write technologies dramatically reduce disk I/O bottlenecks. While some database write whole data block to disk at commit time, Ms-Access commits transactions with at most sequential log file on disk at commit time, On high throughput systems, one sequential writes typically group commit multiple transactions. Data read by the transaction remains as shared memory so that other transactions may access that data without reading it again from disk. Since fast commits write all data necessary to the recovery to the log file, modified blocks are written back to the database independently of the transaction commit, when written from memory to disk.

# CHAPTER 5

# System Design

# SYSTEM DESIGN

## 5.1 INTRODUCTION

System design is the technical phase of the software development life cycle in which the requirements gathered during system analysis are transformed into a structured solution. It defines the architecture, components, interfaces, and data structures of the system.

The system design phase for the **Library Management System** provides a detailed blueprint for implementing the system. It focuses on creating an efficient, reliable, and user-friendly system that meets the functional and non-functional requirements identified earlier.

During this phase, database structures, input and output formats, interface layouts, and processing logic are designed to ensure smooth system operation and ease of maintenance.

**SOFTWARE ENGINEERING PARADIGM APPLIED- (RAD-MODEL)**

The **Rapid Application Development (RAD)** model is adopted for developing the Library Management System. RAD emphasizes rapid prototyping, user feedback, and quick iterations.

### Design Objectives of RAD Model :

- Faster development time
- Improved quality through continuous user feedback
- Better reliability and maintainability

### RELIABLE SYSTEM :

Reliability ensures that the system performs its intended functions accurately and consistently.

### Approaches to Reliability:

1. **Error Avoidance :** Prevents errors by validating user inputs and following structured programming techniques.
2. **Error Detection and Correction :** Detects errors during execution and corrects them without system failure.

3. **Error Tolerance :** Allows the system to continue functioning even if minor errors occur.

## Maintenance:

Maintenance is the process of modifying and updating the system after deployment to improve performance or correct faults.

### Key factors reducing maintenance effort:

1. Clearly defining user requirements.
2. Maintaining proper documentation.
3. Using efficient coding standards.
4. Effective use of development tools.
5. Proper management of system engineering processes.

## Output Design:

Output design is one of the most important aspects of an information system. The quality of output determines the effectiveness of the system.

In the Library Management System, outputs include reports, notifications, and transaction details presented in an organized and readable format.

### Types of Output:

- Printed reports.
- Screen messages.
- Retrieval from database.
- Transaction confirmations.
- System activity outputs.

## Layout Design:

Layout design refers to the arrangement of information on output screens and reports. Proper layout design improves readability and user interaction.

The outputs of the Library Management System are designed to cover all necessary information such as book details, user details, issue/return status, and fine information.

## Input design and control:

Input design focuses on how data is entered into the system. Efficient input design ensures accuracy and reliability of data.

## Objectives of input design:

Input design consists of developing specifications and procedures for data preparation, the steps necessary to put transaction data into a usable from for processing and data entry, the activity of data into the computer processing. The five objectives of input design are:

- Controlling the amount of input.
- Avoiding delays.
- Minimizing input errors.
- Avoiding unnecessary steps.
- Keeping the process simple.

## Controlling the amount of input:

Data preparation and data entry operation depend on people, because labour costs are high, the cost of preparing and entering data is also high. Reducing data requirement expense. By reducing input requirement the speed of entire process from data capturing to processing to provide results to users.

## Avoiding delay:

The processing delay resulting from data preparation or data entry operations is called bottlenecks. Avoiding bottlenecks should be one objective of input.

**Avoiding errors:** Through input validation we control the errors in the input data.

## Avoiding extra steps:

The designer should avoid the input design that cause extra steps in processing saving or adding a single step in large number of transactions saves a lot of processing time or takes more time to process.

## Keeping process simple:

If controls are more people may feel difficult in using the systems. The best-designed system fits the people who use it in a way that is comfortable for them.

## 5.2 NORMALIZATION

Normalization is the process of organizing data in a database to reduce redundancy and improve data integrity.

### Anomalies Addressed by Normalization:

- **Insertion Anomaly** : Difficulty in inserting new data.
- **Deletion Anomaly** : Unintentional loss of data.
- **Updation Anomaly** : Inconsistency in data updates.
- **Normal Forms** : These are the rules for structuring relations that eliminate anomalies.

### First Normal Form (1NF):

A relation is in first normal form if all attribute values are atomic and there are no repeating groups.

### Second Normal Form (2NF):

A relation is in second normal form if:

1. It is in first normal form.
2. No non-key attribute is partially dependent on the primary key.

### Third Normal Form (3NF):

A relation is in third normal form if:

- It is in second normal form.
- There are no transitive dependencies.

### Transitive Dependency:

If non-key attributes depend on other non-key attributes, the relation has a transitive dependency.

The normalization principles have been applied to design the database tables of the Library Management System to ensure consistency and accuracy.

# CHAPTER 6

# E-R Diagram

# E-R DIAGRAM

## 6.1 E-R DIAGRAM

The relationship upon the system is structured through a conceptual **Entity–Relationship Diagram (ER Diagram)**, which specifies the entities involved in the system and the relationships between them. The ER diagram represents the logical structure of the **Library Management System** and describes how data is stored, accessed, and related.

The **Entity Relationship Diagram (ERD)** depicts the relationships between data objects. It is a graphical representation used to model database structure by defining entities, attributes, and relationships.

The primary purpose of the ERD is to represent data objects and their relationships clearly and accurately.

### Primary Components Identified by ERD

- Data Objects (Entities)
- Relationships
- Attributes
- Key Attributes
- Cardinality Constraints

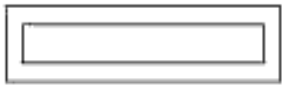
In E-R diagrams, entity types are represented by squares. See the table below. Relationship types are shown in diamond shaped boxes attached to the participating entity types with straight lines. Attributes are shown in ovals, and each attribute is attached to its entity type or relationship type by a straight line. Multivalued attributes are shown in double ovals. Key attributes have their names underlined. Derived attributes are shown in dotted ovals.

Weak entity types are distinguished by being placed in double rectangles and by having their identifying relationship placed in double diamonds.

Attaching a 1, M, or N on each participating edge specifies cardinality ratio of each binary relationship type. The participation constraint is specified by a single line for partial participation and by double lines for total participation. The participation constraints specify whether the existence of an entity depends on its being related to another entity via the relationship type. If every entity of an entity set is related to some other entity set via a relationship type, then the participation of the first entity

type is total. If only few member of an entity type is related to some entity type via a relationship type, the participation is partial.

ENTITY TYPE



WEAK ENTITY TYPE

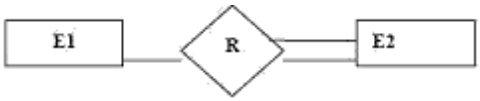
RELATIONSHIP TYPE

ATTRIBUTE

KEY ATTRIBUTE

MULTIVALUED ATTRIBUTE

DERIVED ATTRIBUTE




TOTAL PARTICIPATION OF E2 IN R

Cardinality Ratio 1:N FOR E1:E2 IN R

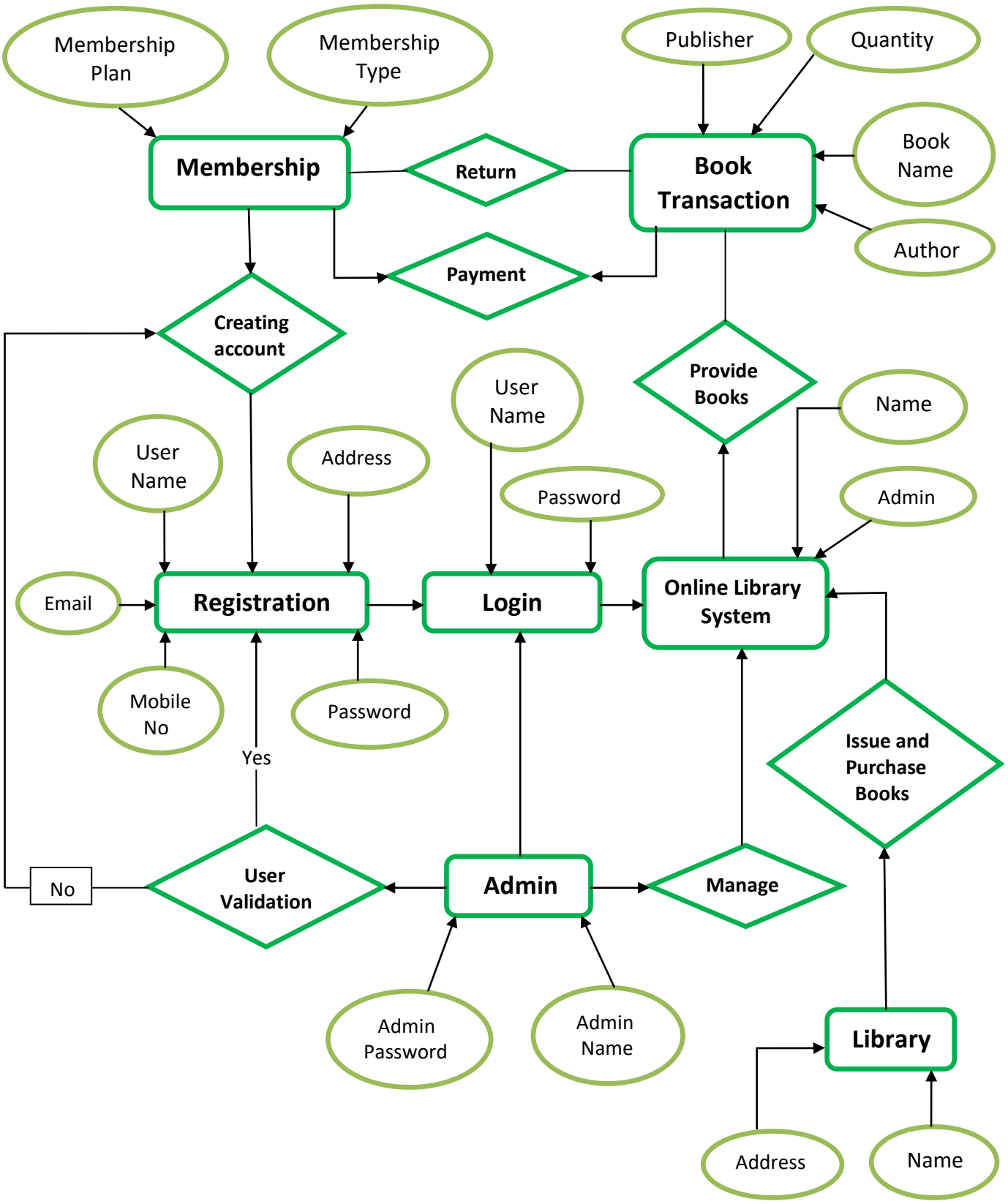
Structural Constraint(Min,Max) On Participation Of E In R

## Naming Data Objects

The names of data objects in the Library Management System follow these rules:

- Names are **unique.**
- Names are **meaningful to the user.**
- Use **minimum number of words.**
- Entity and attribute names are **singular nouns.**
- Relationship names are generally **verbs.**

# ER-DIAGRAM OF LIBRARY MANAGEMENT SYSTEM



# CHPATER 7

# DATA FLOW DIAGRAMS

# DATA FLOW DIAGRAM

## 7.1 DATA FLOW DIAGRAMS

A Data Flow Diagram (DFD) is a graphical tool used to describe and analyze the flow of data through a system. It shows how data is input to a system, processed, stored, and finally output from the system. DFDs focus on the logical flow of information rather than physical implementation.

In the **Library Management System**, the DFD explains how users and administrators interact with the system to perform operations such as registration, login, book search, issue, return, and report generation.

The development of DFDs is done in levels. Each lower-level diagram provides more details than the higher-level diagram.

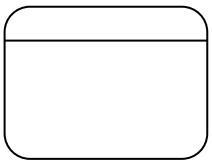
Larry Constantine first developed the DFD as a way of expressing system requirements in a graphical from, this lead to the modular design.

A DFD is also known as a "bubble Chart" has the purpose of clarifying system requirements and identifying major transformations that will become programs in system design. So it is the starting point of the design to the lowest level of detail. A DFD consists of a series of bubbles joined by data flows in the system.

**DFD SYMBOLS:**

In the DFD, there are four symbols :-

1. A square defines a source(originator) or destination of system data
2. An arrow identifies data flow. It is the pipeline through which the information flows
3. A circle or a bubble represents a process that transforms incoming data flow into outgoing data flows.
4. An open rectangle is a data store, data at rest or a temporary repository of data.



Process that transforms data flow.

Source or Destination of data

Data flow

Data Store

**CONSTRUCTING A DFD :**

Several rules of thumb are used in drawing DFD's :

1. Process should be named and numbered for an easy reference. Each name should be representative of the process.
2. The direction of flow is from top to bottom and from left to right. Data traditionally flow from source to the destination although they may flow back to the source. One way to indicate this is to draw long flow line back to a source. An alternative way is to repeat the source symbol as a destination. Since it is used more than once in the DFD it is marked with a short diagonal.
3. When a process is exploded into lower level details, they are numbered.
4. The names of data stores and destinations are written in capital letters. Process and dataflow names have the first letter of each work capitalized

A DFD typically shows the minimum contents of data store. Each data store should contain all the data elements that flow in and out.

Questionnaires should contain all the data elements that flow in and out. Missing interfaces redundancies and like is then accounted for often through interviews.

**SAILENT FEATURES OF DFD's :**

1. The DFD shows flow of data, not of control loops and decision are controlled considerations do not appear on a DFD.
2. The DFD does not indicate the time factor involved in any process whether the data flows take place daily, weekly, monthly or yearly.
3. The sequence of events is not brought out on the DFD.

## RULES GOVERNING THE DFD'S

### PROCESS

1. No process can have only outputs.
2. No process can have only inputs.
3. Every process must have a meaningful verb phrase label.

### DATA STORE

1. Data cannot move directly from one data store to another.
2. Data must pass through a process before entering a data store.
3. A data store has a noun phrase label.

### SOURCE OR SINK

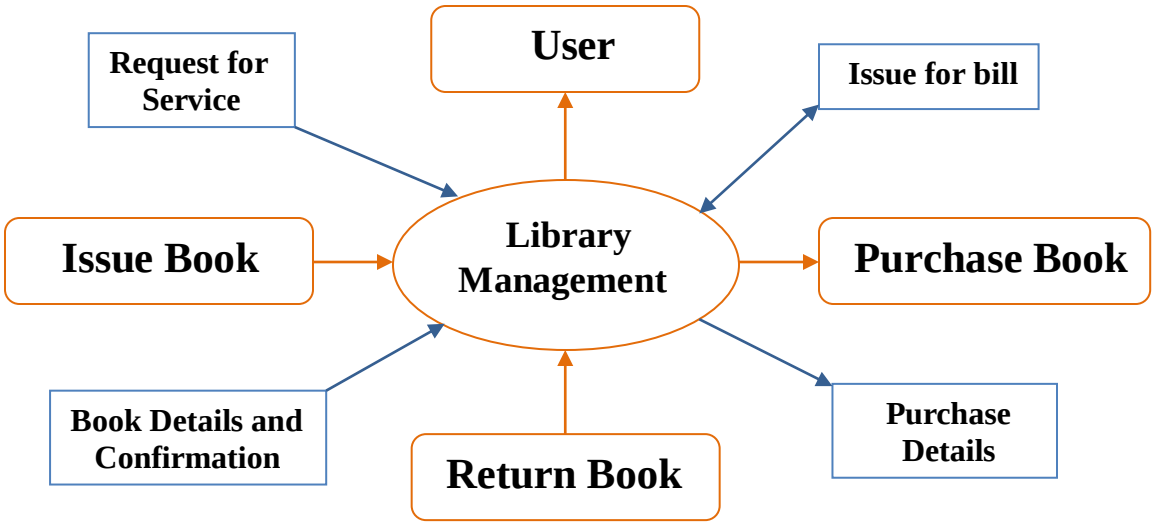
1. Data cannot move directly from source to sink.
2. A source or sink has a noun phrase label.

### DATA FLOW

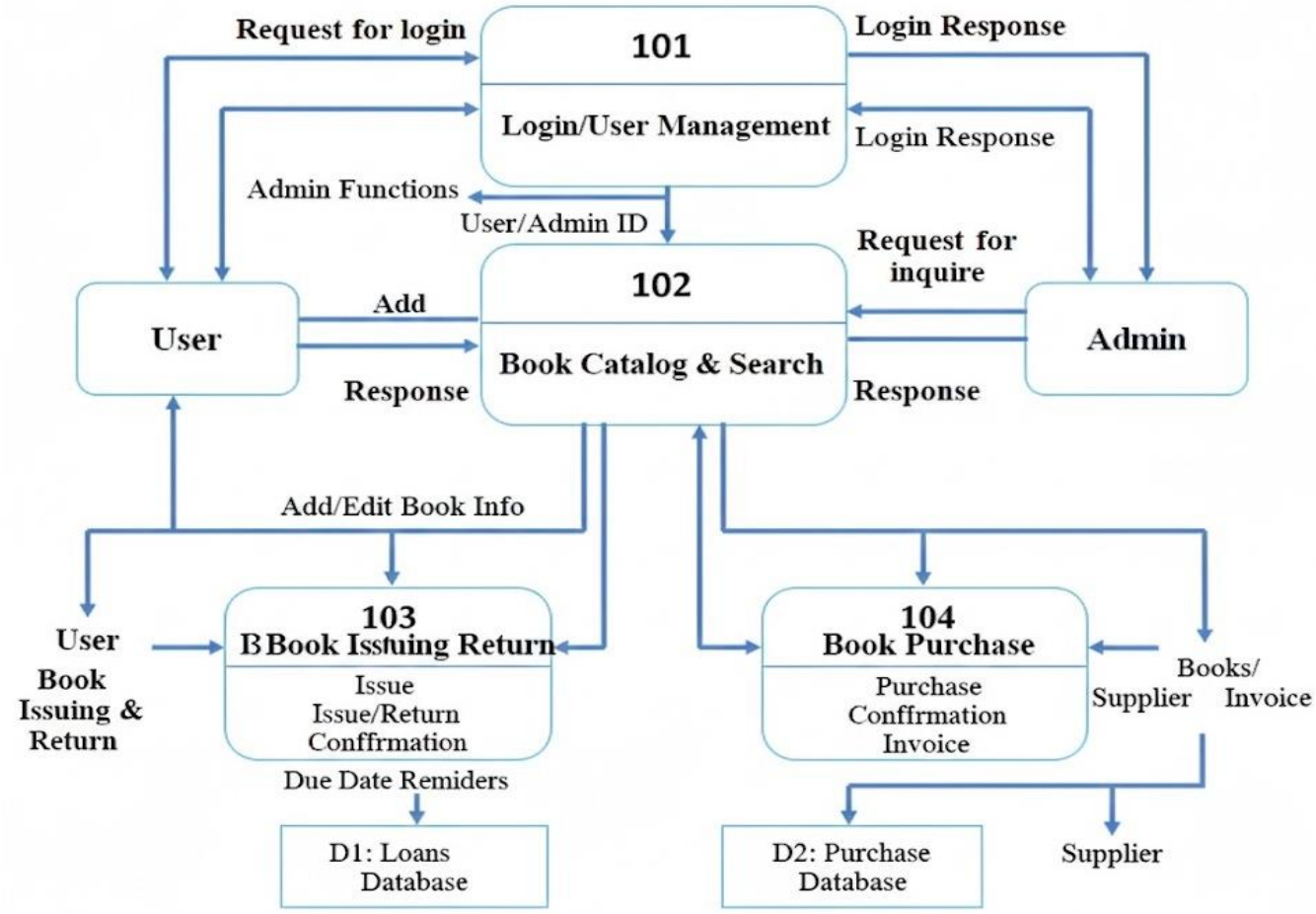
1. A Data Flow has only one direction of flow between symbol. It may flow in both directions between a process and a data store to show a read before an update. The later is usually indicated however by two separate arrows since these happen at different type.
2. A join in DFD means that exactly the same data comes from any of two or more different processes data store or sink to a common location.
3. A data flow cannot go directly back to the same process it leads. There must be atleast one other process that handles the data flow produce some other data flow returns the original data into the beginning process.
4. A Data flow to a data store means update ( delete or change).
5. A data Flow from a data store means retrieve or use.

   A data flow has a noun phrase label more than one data flow noun phrase can appear on a single arrow as long as all of the flows on the same arrow move together as one package.

# DFD Of Library Management System




**Data Flow Diagram Level 0: Library Management System**




**Data Flow Diagram Level 1: Library Management System**

# CHAPTER 8

# Data Base Design

# DATA BASE DESIGN

## 8.1 DATA DICTIONARY

After carefully understanding the requirements of the client the entire data storage requirements are divided into tables. The below tables are normalized to avoid any anomalies during the course of data entry.

### 1. Registration

Register

| Field Name | Data Type |
|---|---|
| MemberID | AutoNumber |
| Username | Text |
| Password | Text |
| Course | Text |
| EMail | Text |
| MobileNo | Text |
| RegistrationDate | Date/Time |

### 2. Membership

Membership

| Field Name | Data Type |
|---|---|
| MemberID | Text |
| Name | Text |
| DOB | Date/Time |
| Gender | Text |
| Email | Text |
| Mobile | Text |
| Address | Text |
| MembershipType | Text |
| MembershipPlan | Text |
| RegistrationDate | Text |
| ExpiryDate | Text |
| Fees | Currency |
| PaymentMethod | Text |

## 3. Book Details

BookDetail

| Field Name | Data Type |
|---|---|
| BookID | AutoNumber |
| Title | Text |
| Author | Text |
| Publisher | Text |
| BookYear | Text |
| AvailableCopies | Number |
| Price | Number |
| ResourceName | Text |

## 4. Purchase Book Details

PurchaseBook

| Field Name | Data Type |
|---|---|
| InvoiceNo | Text |
| PurchaseID | AutoNumber |
| UserName | Text |
| BookTitle | Text |
| BookAuthor | Text |
| PurchaseDate | Date/Time |
| BookPrice | Text |
| PaymentMethod | Text |
| Status | Text |

## 5. Issue Book Details

IssueBook

| Field Name | Data Type |
|---|---|
| InvoiceNo | Text |
| UserName | Text |
| BookTitle | Text |
| BookAuthor | Text |
| IssueDate | Date/Time |
| Usertype | Text |
| UserMembership | Text |
| Status | Text |

## 6. Return Book Details

ReturnBook

| Field Name | Data Type |
|---|---|
| UserName | Text |
| BookTitle | Text |
| BookAuthor | Text |
| ReturnDate | Date/Time |
| Usertype | Text |
| UserMembership | Text |
| Status | Text |

## 7. Overdue and Fine

Overdue&Fine

| Field Name | Data Type |
|---|---|
| InvoiceNo | Text |
| Username | Text |
| OverdueDays | Text |
| FineAmount | Text |
| PaymentMethod | Text |
| PaidDate | Date/Time |

# CHAPTER 9

# Input & Output Screen Design

# OUTPUT SCREEN (SCREENSHOTS)

**Design Interface of Our Program**

- The user interface of the Library Management System is simple, user-friendly, and easy to understand.
- All screens are designed to allow users and administrators to perform operations efficiently.
- The design of the interfaces is shown below with proper explanations.

## 1. First Page :

➢ This is the first page of the Library Management System. Here user can access Login, Registration, About and Contact. Existing users can log in using their Username and Password.

## 2. User Login Authentication :

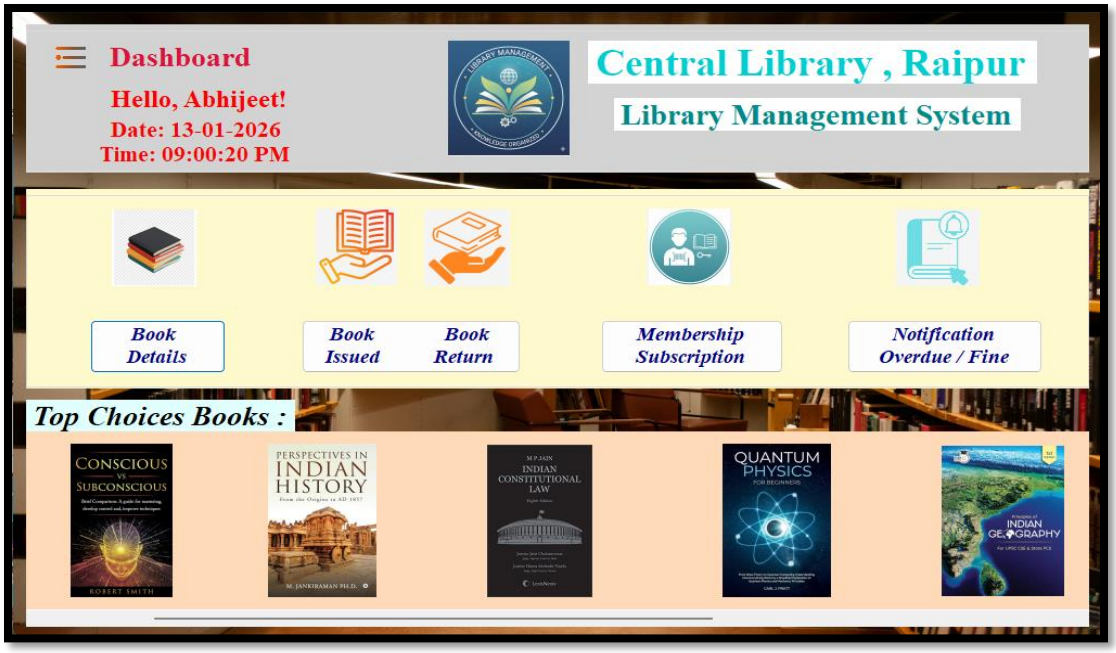
- This screen verifies user crendentials. In this section if User entered correct Username or Password then the home Form of the software will be displayed, otherwise an error message will be displayed.

## 3. Registration :

- This screen allows new user to register for the Library Management System. In this form user need input details are Username, Password, Confirm Password, Course, Email-ID, Mobile No and Registration Date.

# 4. Dashboard :

- This screen displays available books and options after login.




# 5. Membership Subscription :

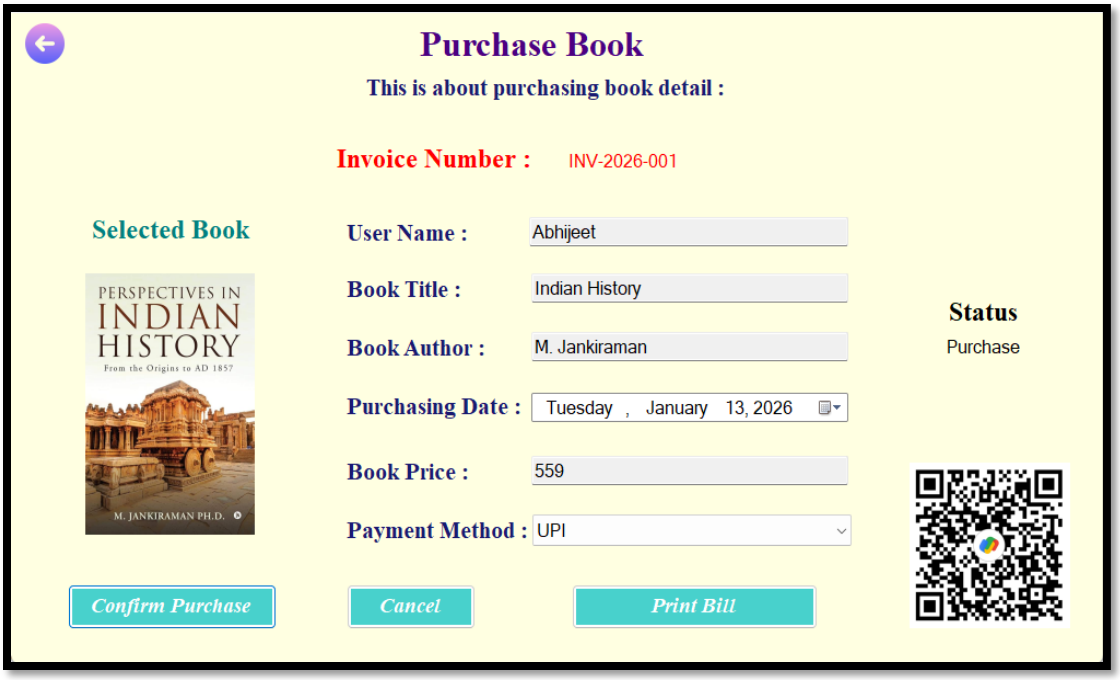
- This screen allows users to take a membership subscription before issuing books.

## 6. Book Details :

- This screen allows users to select their own book and user can issue and purchase those books.

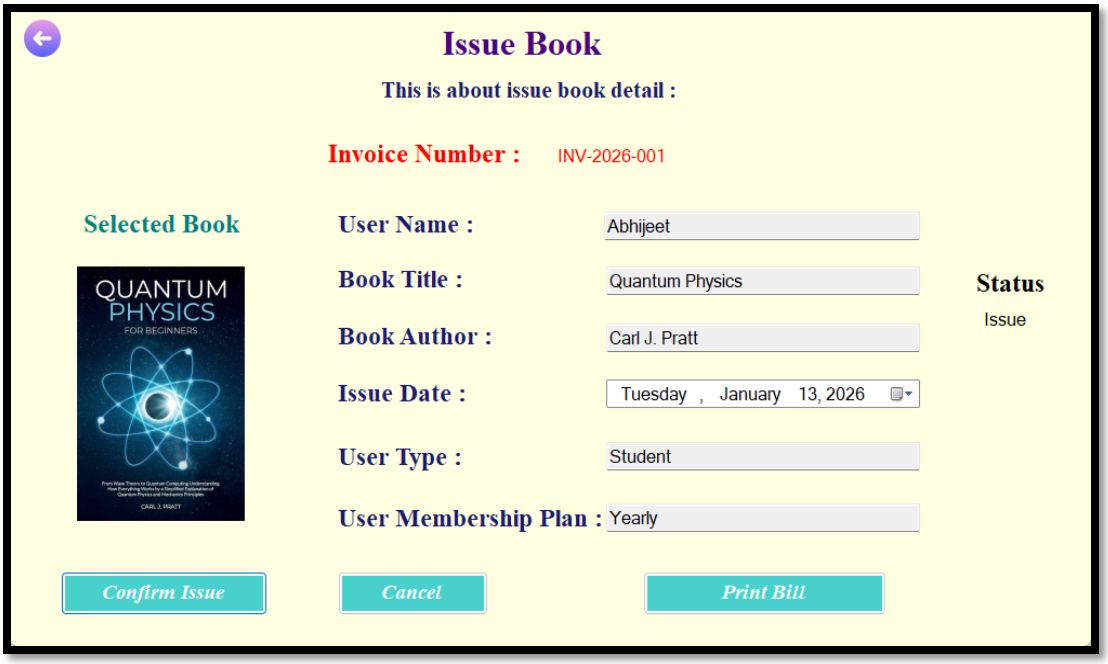
## 7. Purchase Books :

- This screen allows users to purchase selected books directly.



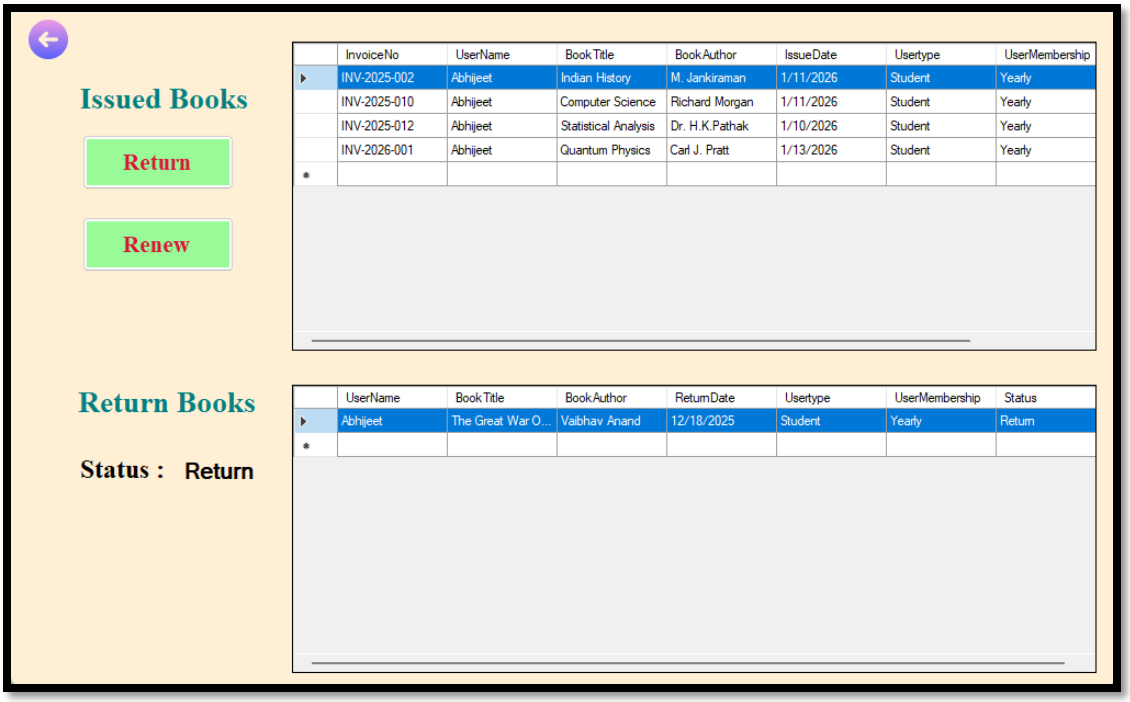
# 8. Issue Books :

- This screen is used to issue selected books to users.



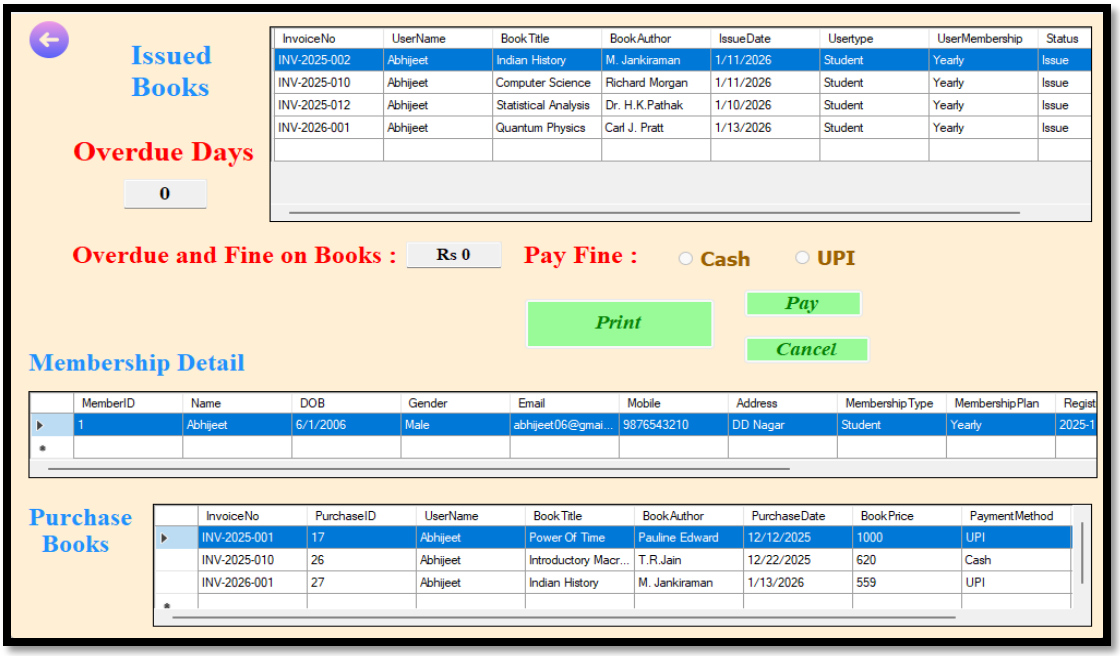

# 9. Return and Renew Books :

- This screen allow the users to renew and return the issue books.



# 10. Notification Overdue and Fine on Books :

- This screen calculates fine if the book is returned after 14 days and also confirm fine payment.



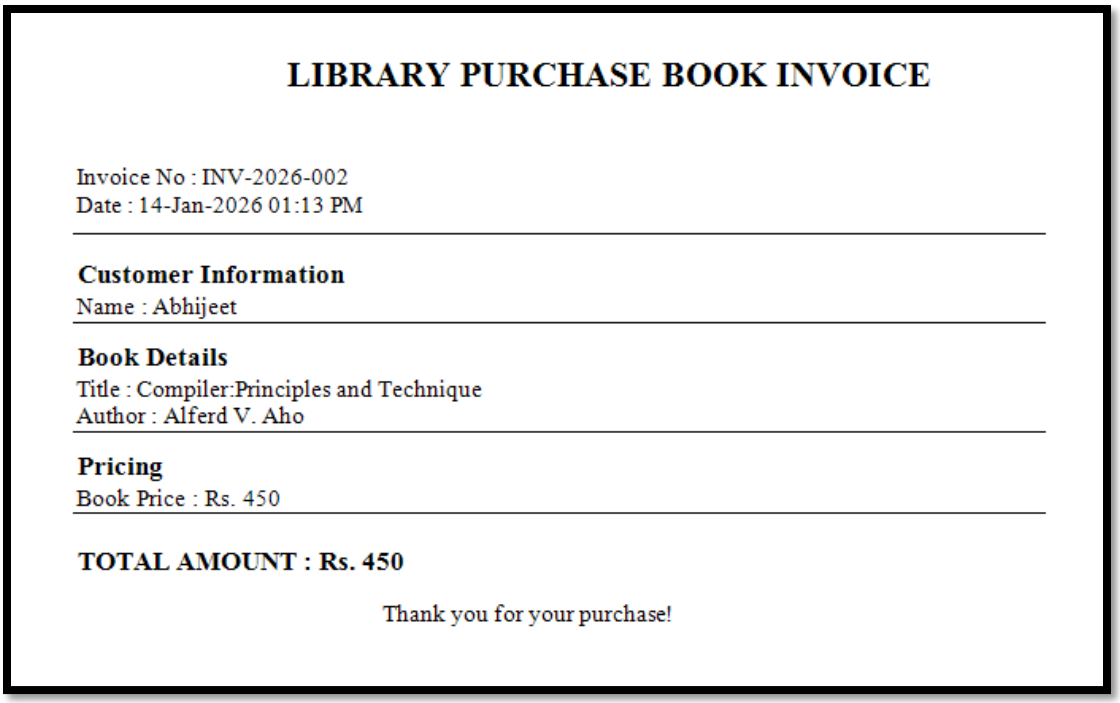

| InvoiceNo | UserName | BookTitle | BookAuthor | IssueDate | Usertype | UserMembership | Status |
|---|---|---|---|---|---|---|---|
| INV-2025-002 | Abhijeet | Indian History | M. Jankiraman | 1/11/2026 | Student | Yearly | Issue |
| INV-2025-010 | Abhijeet | Computer Science | Richard Morgan | 1/11/2026 | Student | Yearly | Issue |
| INV-2025-012 | Abhijeet | Statistical Analysis | Dr. H.K.Pathak | 1/10/2026 | Student | Yearly | Issue |
| INV-2026-001 | Abhijeet | Quantum Physics | Carl J. Pratt | 1/13/2026 | Student | Yearly | Issue |

| MemberID | Name | DOB | Gender | Email | Mobile | Address | MembershipType | MembershipPlan | Regist |
|---|---|---|---|---|---|---|---|---|---|
| 1 | Abhijeet | 6/1/2006 | Male | abhijeet06@gmai... | 9876543210 | DD Nagar | Student | Yearly | 2025-1 |

| InvoiceNo | PurchaseID | UserName | BookTitle | BookAuthor | PurchaseDate | BookPrice | PaymentMethod |
|---|---|---|---|---|---|---|---|
| INV-2025-001 | 17 | Abhijeet | Power Of Time | Pauline Edward | 12/12/2025 | 1000 | UPI |
| INV-2025-010 | 26 | Abhijeet | Introductory Macr... | T.R.Jain | 12/22/2025 | 620 | Cash |
| INV-2026-001 | 27 | Abhijeet | Indian History | M. Jankiraman | 1/13/2026 | 559 | UPI |

# 11. Invoice Receipt :

- On this page user will receive Bill receipt.



**LIBRARY PURCHASE BOOK INVOICE**

Invoice No : INV-2026-002
Date : 14-Jan-2026 01:13 PM

**Customer Information**
Name : Abhijeet

**Book Details**
Title : Compiler:Principles and Technique
Author : Alferd V. Aho

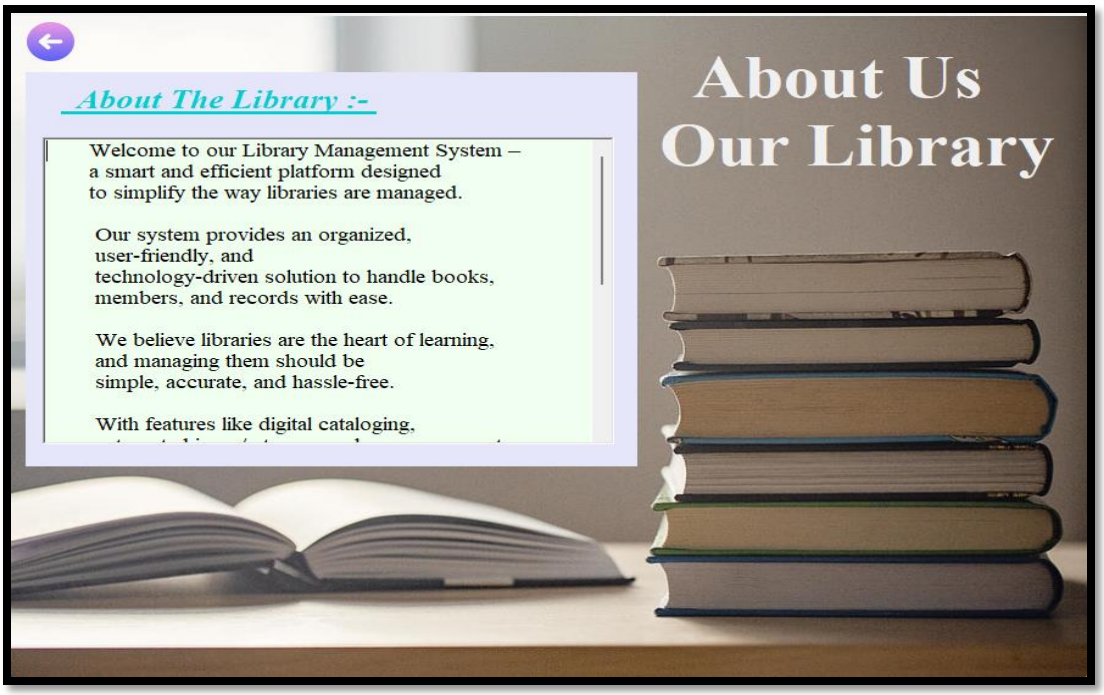
**Pricing**
Book Price : Rs. 450

**TOTAL AMOUNT : Rs. 450**

Thank you for your purchase!

## 12. About :

- On this page user can get information about Library.



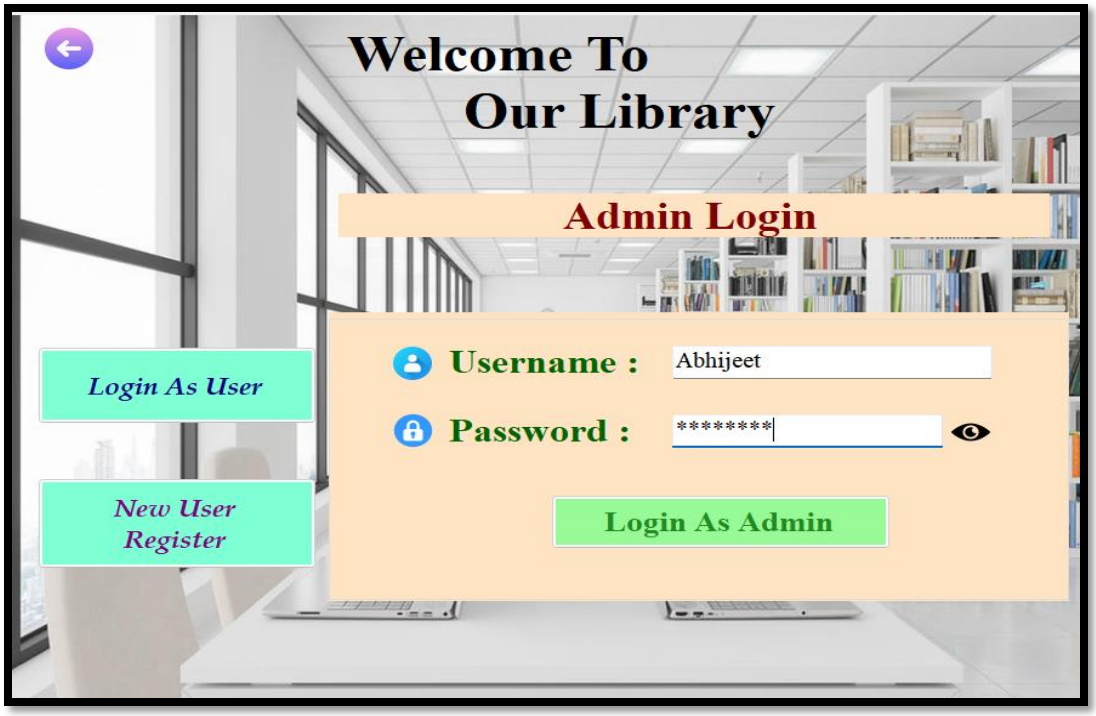

## 13. Contact :

- This page allows users to contact the library administration for queries, support, and feedback through E-Mail or Phone.

# Admin Form

## 14. Admin Login Authentication :

- This screen verifies Admin crendentials.

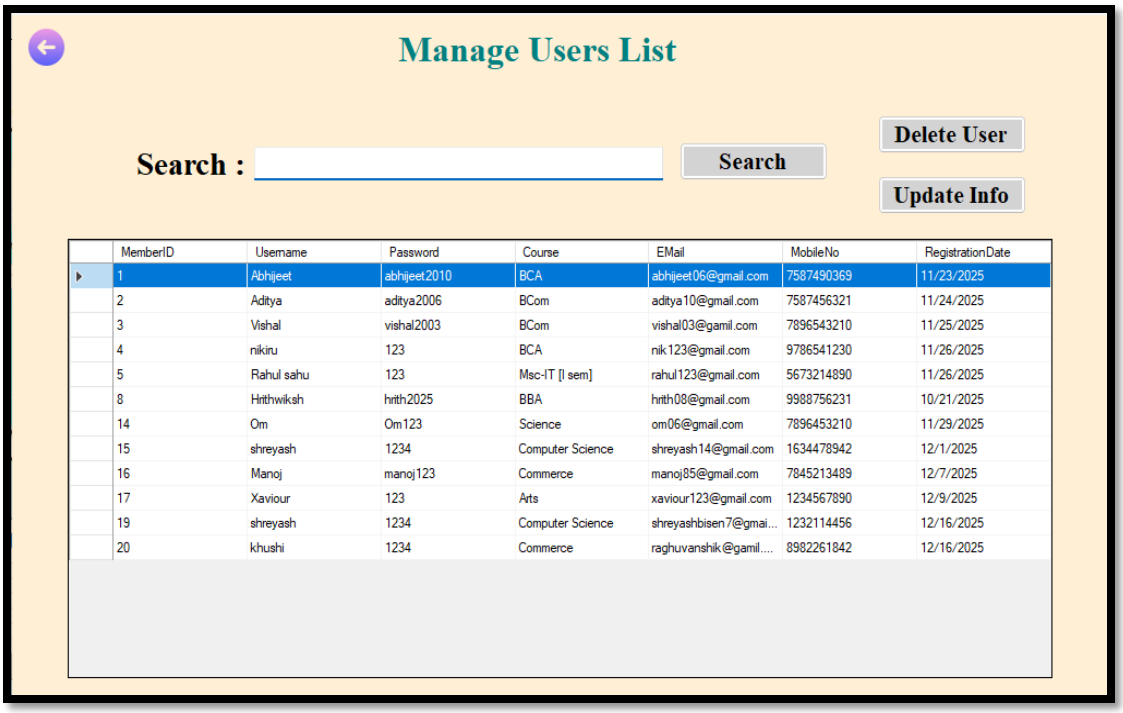



## 15. Admin Profile :

# 16. Admin Main Page :

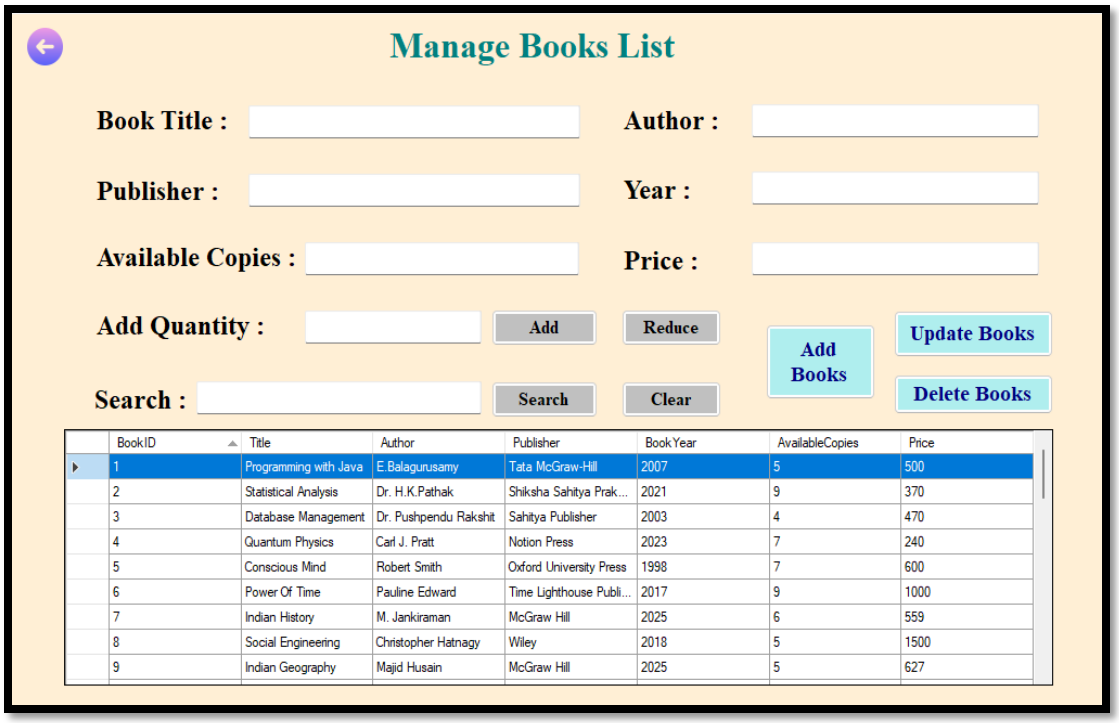
- In this page Admin can see and manage all Registration, Membership, Books, Issue and Return Book history, Purchase Books and Fine Details.

# 17. Users :



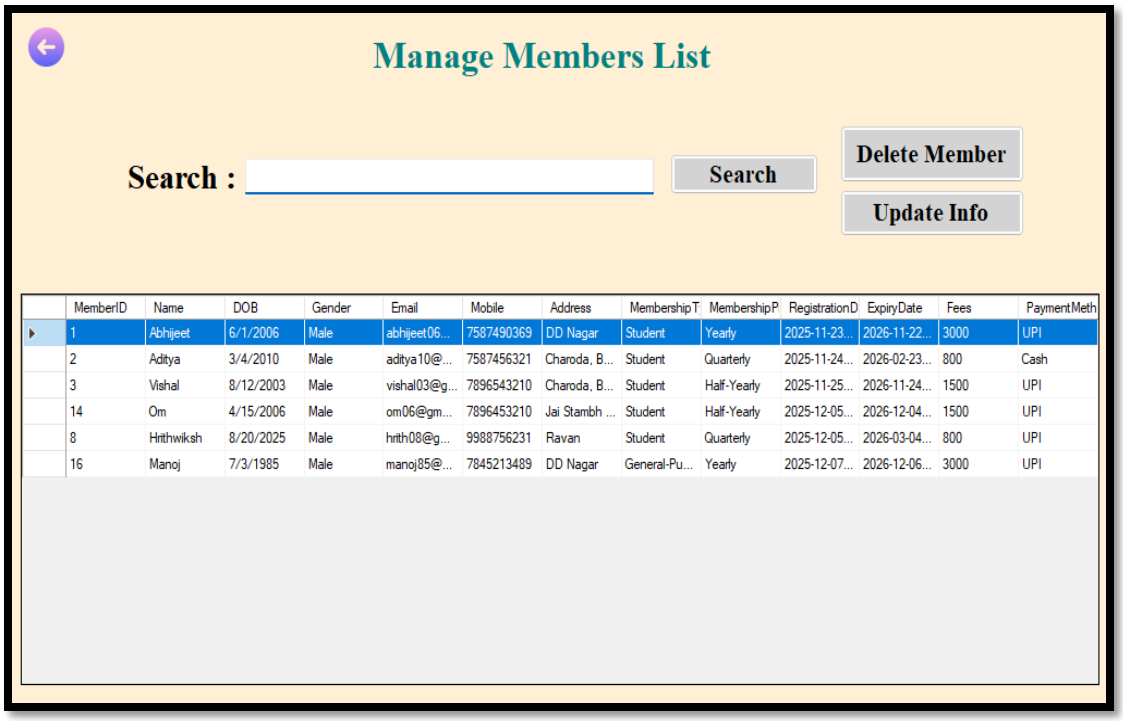

| MemberID | Username | Password | Course | EMail | MobileNo | RegistrationDate |
|---|---|---|---|---|---|---|
| 1 | Abhijeet | abhijeet2010 | BCA | abhijeet06@gmail.com | 7587490369 | 11/23/2025 |
| 2 | Aditya | aditya2006 | BCom | aditya10@gmail.com | 7587456321 | 11/24/2025 |
| 3 | Vishal | vishal2003 | BCom | vishal03@gamil.com | 7896543210 | 11/25/2025 |
| 4 | nikiru | 123 | BCA | nik123@gmail.com | 9786541230 | 11/26/2025 |
| 5 | Rahul sahu | 123 | Msc-IT [I sem] | rahul123@gmail.com | 5673214890 | 11/26/2025 |
| 8 | Hrithwiksh | hrith2025 | BBA | hrith08@gmail.com | 9988756231 | 10/21/2025 |
| 14 | Om | Om123 | Science | om06@gmail.com | 7896453210 | 11/29/2025 |
| 15 | shreyash | 1234 | Computer Science | shreyash14@gmail.com | 1634478942 | 12/1/2025 |
| 16 | Manoj | manoj123 | Commerce | manoj85@gmail.com | 7845213489 | 12/7/2025 |
| 17 | Xaviour | 123 | Arts | xaviour123@gmail.com | 1234567890 | 12/9/2025 |
| 19 | shreyash | 1234 | Computer Science | shreyashbisen7@gmai... | 1232114456 | 12/16/2025 |
| 20 | khushi | 1234 | Commerce | raghuvanshik@gamil.... | 8982261842 | 12/16/2025 |

# 18. Books :



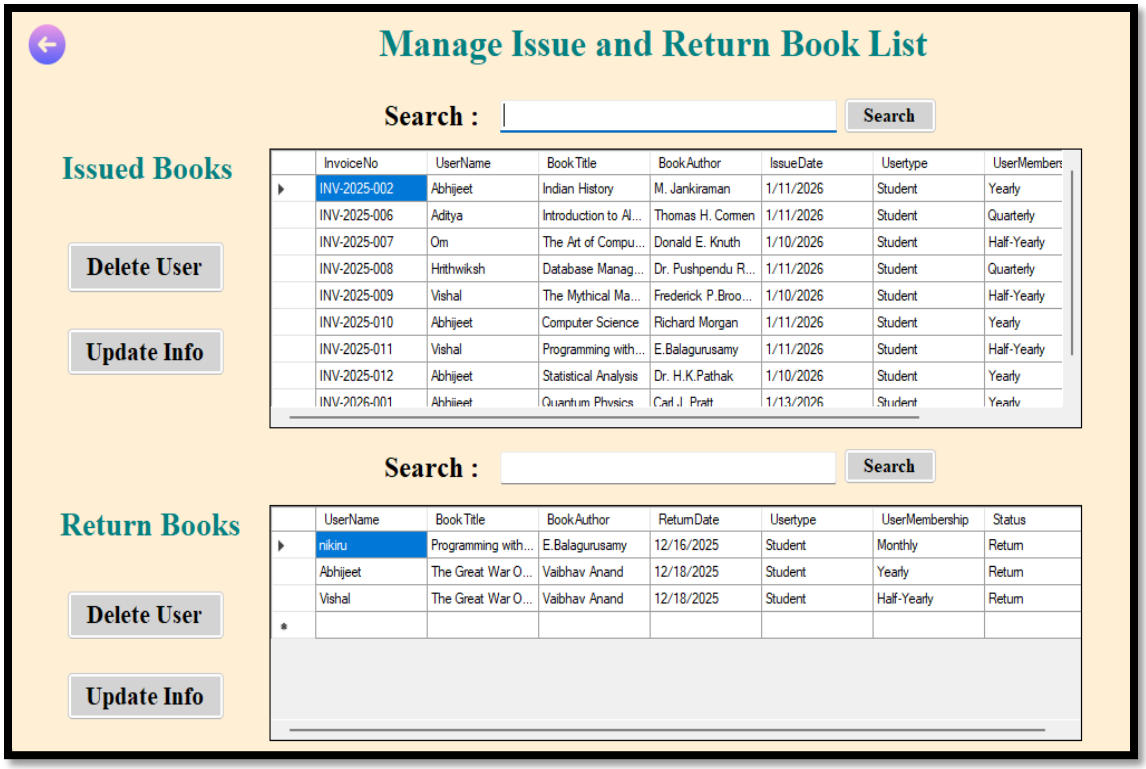

| BookID | Title | Author | Publisher | BookYear | AvailableCopies | Price |
|---|---|---|---|---|---|---|
| 1 | Programming with Java | E.Balagurusamy | Tata McGraw-Hill | 2007 | 5 | 500 |
| 2 | Statistical Analysis | Dr. H.K.Pathak | Shiksha Sahitya Prak... | 2021 | 9 | 370 |
| 3 | Database Management | Dr. Pushpendu Rakshit | Sahitya Publisher | 2003 | 4 | 470 |
| 4 | Quantum Physics | Carl J. Pratt | Notion Press | 2023 | 7 | 240 |
| 5 | Conscious Mind | Robert Smith | Oxford University Press | 1998 | 7 | 600 |
| 6 | Power Of Time | Pauline Edward | Time Lighthouse Publi... | 2017 | 9 | 1000 |
| 7 | Indian History | M. Jankiraman | McGraw Hill | 2025 | 6 | 559 |
| 8 | Social Engineering | Christopher Hatnagy | Wiley | 2018 | 5 | 1500 |
| 9 | Indian Geography | Majid Husain | McGraw Hill | 2025 | 5 | 627 |

# 19. Membership :




| MemberID | Name | DOB | Gender | Email | Mobile | Address | MembershipT | MembershipP | RegistrationD | ExpiryDate | Fees | PaymentMeth |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 1 | Abhijeet | 6/1/2006 | Male | abhijeet06... | 7587490369 | DD Nagar | Student | Yearly | 2025-11-23... | 2026-11-22... | 3000 | UPI |
| 2 | Aditya | 3/4/2010 | Male | aditya10@... | 7587456321 | Charoda, B... | Student | Quarterly | 2025-11-24... | 2026-02-23... | 800 | Cash |
| 3 | Vishal | 8/12/2003 | Male | vishal03@g... | 7896543210 | Charoda, B... | Student | Half-Yearly | 2025-11-25... | 2026-11-24... | 1500 | UPI |
| 14 | Om | 4/15/2006 | Male | om06@gm... | 7896453210 | Jai Stambh ... | Student | Half-Yearly | 2025-12-05... | 2026-12-04... | 1500 | UPI |
| 8 | Hrithwiksh | 8/20/2025 | Male | hrith08@g... | 9988756231 | Ravan | Student | Quarterly | 2025-12-05... | 2026-03-04... | 800 | UPI |
| 16 | Manoj | 7/3/1985 | Male | manoj85@... | 7845213489 | DD Nagar | General-Pu... | Yearly | 2025-12-07... | 2026-12-06... | 3000 | UPI |

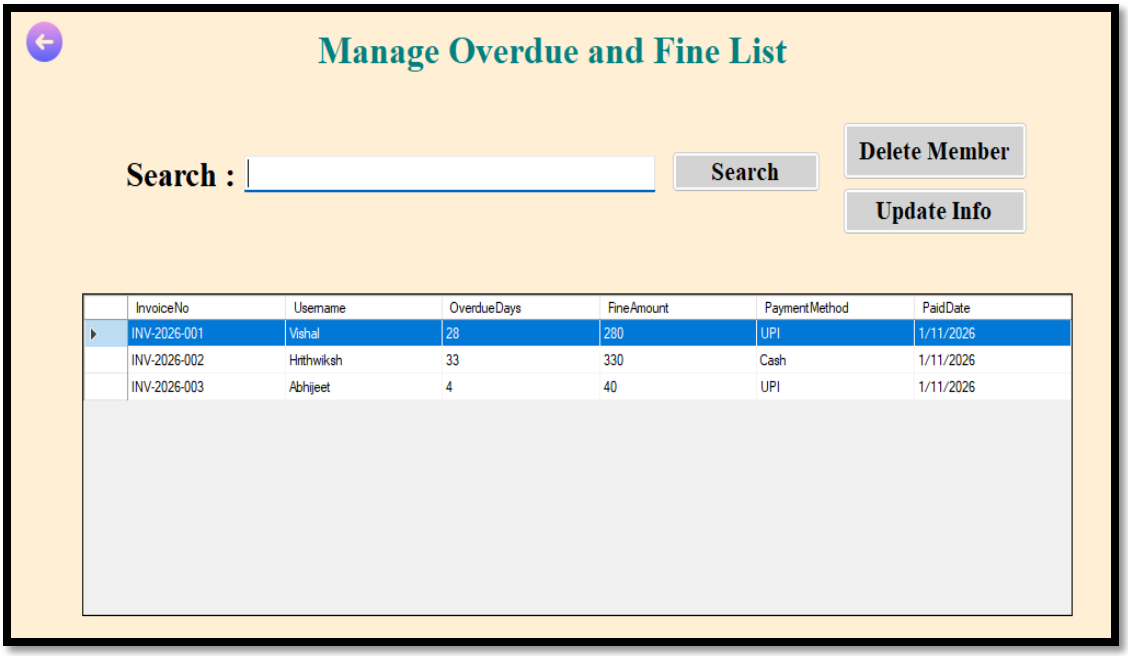
## 20. Issue and Return Books :




| InvoiceNo | UserName | BookTitle | BookAuthor | IssueDate | Usertype | UserMembers |
|---|---|---|---|---|---|---|
| INV-2025-002 | Abhijeet | Indian History | M. Jankiraman | 1/11/2026 | Student | Yearly |
| INV-2025-006 | Aditya | Introduction to AI... | Thomas H. Cormen | 1/11/2026 | Student | Quarterly |
| INV-2025-007 | Om | The Art of Compu... | Donald E. Knuth | 1/10/2026 | Student | Half-Yearly |
| INV-2025-008 | Hrithwiksh | Database Manag... | Dr. Pushpendu R... | 1/11/2026 | Student | Quarterly |
| INV-2025-009 | Vishal | The Mythical Ma... | Frederick P.Broo... | 1/10/2026 | Student | Half-Yearly |
| INV-2025-010 | Abhijeet | Computer Science | Richard Morgan | 1/11/2026 | Student | Yearly |
| INV-2025-011 | Vishal | Programming with... | E.Balagurusamy | 1/11/2026 | Student | Half-Yearly |
| INV-2025-012 | Abhijeet | Statistical Analysis | Dr. H.K.Pathak | 1/10/2026 | Student | Yearly |
| INV-2026-001 | Abhijeet | Quantum Physics | Carl J. Pratt | 1/13/2026 | Student | Yearly |

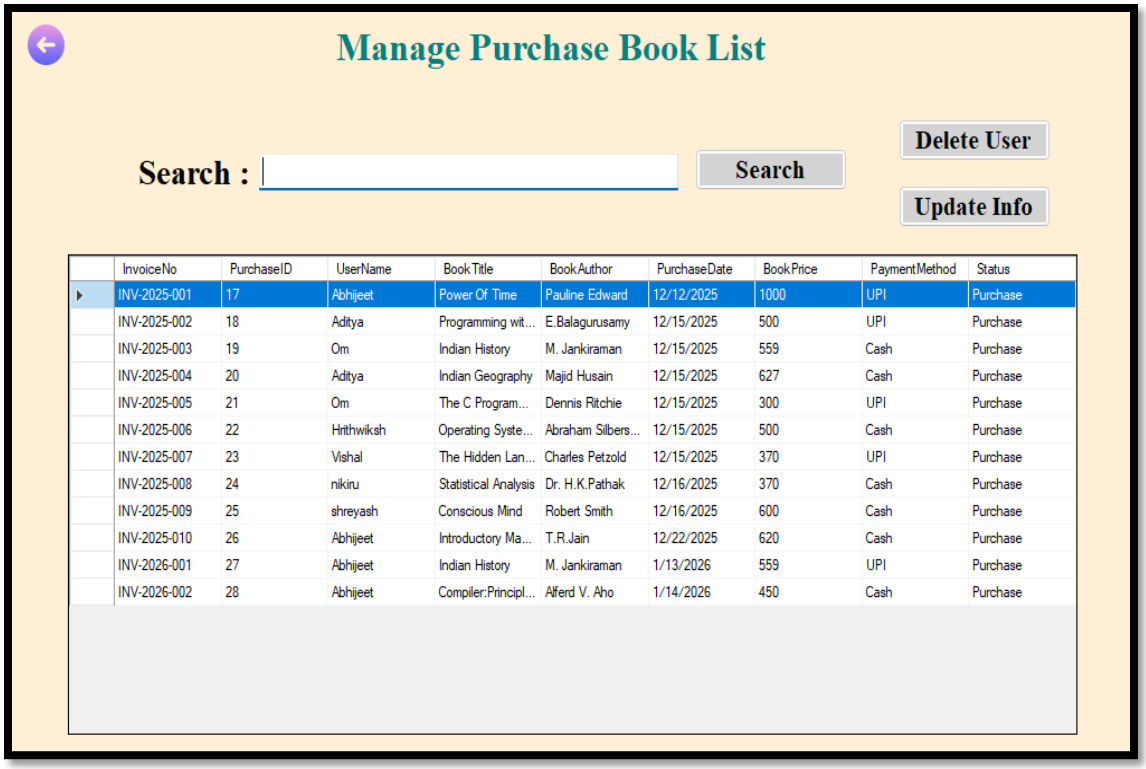
| UserName | BookTitle | BookAuthor | ReturnDate | Usertype | UserMembership | Status |
|---|---|---|---|---|---|---|
| nikiru | Programming with... | E.Balagurusamy | 12/16/2025 | Student | Monthly | Return |
| Abhijeet | The Great War O... | Vaibhav Anand | 12/18/2025 | Student | Yearly | Return |
| Vishal | The Great War O... | Vaibhav Anand | 12/18/2025 | Student | Half-Yearly | Return |

## 21. Overdue and Fine :




| InvoiceNo | Username | OverdueDays | FineAmount | PaymentMethod | PaidDate |
|---|---|---|---|---|---|
| INV-2026-001 | Vishal | 28 | 280 | UPI | 1/11/2026 |
| INV-2026-002 | Hrithwiksh | 33 | 330 | Cash | 1/11/2026 |
| INV-2026-003 | Abhijeet | 4 | 40 | UPI | 1/11/2026 |

# 22. Purchase Books :




| InvoiceNo | PurchaseID | UserName | BookTitle | BookAuthor | PurchaseDate | BookPrice | PaymentMethod | Status |
|---|---|---|---|---|---|---|---|---|
| INV-2025-001 | 17 | Abhijeet | Power Of Time | Pauline Edward | 12/12/2025 | 1000 | UPI | Purchase |
| INV-2025-002 | 18 | Aditya | Programming wit... | E.Balagurusamy | 12/15/2025 | 500 | UPI | Purchase |
| INV-2025-003 | 19 | Om | Indian History | M. Jankiraman | 12/15/2025 | 559 | Cash | Purchase |
| INV-2025-004 | 20 | Aditya | Indian Geography | Majid Husain | 12/15/2025 | 627 | Cash | Purchase |
| INV-2025-005 | 21 | Om | The C Program... | Dennis Ritchie | 12/15/2025 | 300 | UPI | Purchase |
| INV-2025-006 | 22 | Hrithwiksh | Operating Syste... | Abraham Silbers... | 12/15/2025 | 500 | Cash | Purchase |
| INV-2025-007 | 23 | Vishal | The Hidden Lan... | Charles Petzold | 12/15/2025 | 370 | UPI | Purchase |
| INV-2025-008 | 24 | nikiru | Statistical Analysis | Dr. H.K.Pathak | 12/16/2025 | 370 | Cash | Purchase |
| INV-2025-009 | 25 | shreyash | Conscious Mind | Robert Smith | 12/16/2025 | 600 | Cash | Purchase |
| INV-2025-010 | 26 | Abhijeet | Introductory Ma... | T.R.Jain | 12/22/2025 | 620 | Cash | Purchase |
| INV-2026-001 | 27 | Abhijeet | Indian History | M. Jankiraman | 1/13/2026 | 559 | UPI | Purchase |
| INV-2026-002 | 28 | Abhijeet | Compiler:Principl... | Alferd V. Aho | 1/14/2026 | 450 | Cash | Purchase |

# CHAPTER 10

# CODING

# 1. Login :

```
Imports System.Data.OleDb
Public Class User
    Public Shared LoggedUserName As String
    Public ReceivedUser As String
    Dim con As New OleDbConnection("Provider = Microsoft.ACE.OLEDB.12.0; Data
                                    Source = LibraryManagementDB.accdb")
    Dim cmd As OleDbCommand
    Dim dr As OleDbDataReader
    Dim showPassword As Boolean = True

Private Sub PictureBox2_Click(sender As Object, e As EventArgs) Handles
                                                    PictureBox2.Click
    If showPassword Then
           PictureBox2.Image = My.Resources.eye
           Me.txtPassword.PasswordChar = Global.Microsoft.VisualBasic.ChrW(42)
    Else
           PictureBox2.Image = My.Resources.HIde
           Me.txtPassword.PasswordChar = Nothing
    End If
    showPassword = Not showPassword
 End Sub

Private Sub Button1_Click_1(sender As Object, e As EventArgs) Handles
                                                    Button1.Click
    If txtUsername.Text = "" Or txtPassword.Text = "" Then
            MessageBox.Show("Please enter Username and Password")
    Exit Sub
    End If

    Try
    con.Open()
    Dim dr As OleDbDataReader
    cmd = New OleDbCommand("SELECT * FROM Register WHERE Username=@u AND
                                                    Password=@p", con)
    cmd.Parameters.AddWithValue("@u", txtUsername.Text)
    cmd.Parameters.AddWithValue("@p", txtPassword.Text)
    dr = cmd.ExecuteReader()

    If dr.Read() Then
      User.LoggedUserName = dr("Username").ToString()
      MessageBox.Show("Login Successful ! Welcome " & User.LoggedUserName)
      dr.Close()
      con.Close()
      UserInterface.Show()
      Me.Close()
    Else
      MessageBox.Show("Invalid Username or Password", "Login Failed")
    End If

    Catch ex As Exception
       MessageBox.Show(ex.Message)
    Finally
      If dr IsNot Nothing Then dr.Close()
      If con.State = ConnectionState.Open Then con.Close()
    End Try
    End Sub
 End Class
```

## 2. Registration :

```
Imports System.Data.OleDb
Imports System.Text.RegularExpressions
Public Class Register
Dim con As New OleDbConnection("Provider = Microsoft.ACE.OLEDB.12.0; Data
    Source = F:\Project (Library Management)\My Project (Library
    Management)\Project\bin\Debug\LibraryManagementDB.accdb; Persist Security
    Info=False;")
Dim showPassword As Boolean = True
Private Sub PictureBox2_Click(sender As Object, e As EventArgs) Handles
                                                    PictureBox2.Click
    If showPassword Then
        PictureBox2.Image = My.Resources.eye
        Me.txtPassword.PasswordChar = Global.Microsoft.VisualBasic.ChrW(42)
    Else
        PictureBox2.Image = My.Resources.HIde
        Me.txtPassword.PasswordChar = Nothing
    End If
        showPassword = Not showPassword
    End Sub
 Private Sub PictureBox4_Click(sender As Object, e As EventArgs) Handles
                                                PictureBox4.Click
    If showPassword Then
     PictureBox4.Image = My.Resources.eye
     Me.txtConfirmPassword.PasswordChar = Global.Microsoft.VisualBasic.ChrW(42)
    Else
      PictureBox4.Image = My.Resources.HIde
      Me.txtConfirmPassword.PasswordChar = Nothing
    End If
      showPassword = Not showPassword
    End Sub

Public Function IsValidEmail(email As String) As Boolean
    Return Regex.IsMatch(email, "^[\w\.-]+@[\w\.-]+\.\w+$")
End Function
Public Function IsValidMobile(mobile As String) As Boolean
    Return Regex.IsMatch(mobile, "^[0-9]{10}$")
End Function

Private Sub btnRegister_Click(sender As Object, e As EventArgs) Handles
                                                    btnRegister.Click
    If txtUser.Text.Trim = "" Or txtPassword.Text.Trim = "" Or
    txtConfirmPassword.Text.Trim = "" Or cmbCourse.Text.Trim = "" Or
    txtEmail.Text.Trim = "" Or txtMobile.Text.Trim = "" Then
         MessageBox.Show("All fields are required!", "Error",
                 MessageBoxButtons.OK, MessageBoxIcon.Error)
    Exit Sub
    End If
    If txtPassword.Text <> txtConfirmPassword.Text Then
            MessageBox.Show("Password & Confirm Password must be same!",
                 "Error", MessageBoxButtons.OK, MessageBoxIcon.Error)
    Exit Sub
    End If
    If Not IsValidEmail(txtEmail.Text) Then
           MessageBox.Show("Invalid Email Format!", "Error",
                 MessageBoxButtons.OK, MessageBoxIcon.Error)
```

```
        Exit Sub
        End If
        If Not IsValidMobile(txtMobile.Text) Then
                MessageBox.Show("Mobile number must be 10 digits!", "Error",
                             MessageBoxButtons.OK, MessageBoxIcon.Error)
        Exit Sub
        End If

        Try
             con.Open()
             Dim query As String = "INSERT INTO Register (Username, [Password],
                 Course, Email, MobileNo, RegistrationDate) VALUES (@un, @pw, @cr,
                                                                   @em, @mo, @dt)"

             Dim cmd As New OleDbCommand(query, con)

               cmd.Parameters.AddWithValue("@un", txtUser.Text)
               cmd.Parameters.AddWithValue("@pw", txtPassword.Text)
               cmd.Parameters.AddWithValue("@cr", cmbCourse.Text)
               cmd.Parameters.AddWithValue("@em", txtEmail.Text)
               cmd.Parameters.AddWithValue("@mo", txtMobile.Text)
               cmd.Parameters.AddWithValue("@dt", dtpRegister.Value.Date)
               cmd.ExecuteNonQuery()

               MessageBox.Show("Registration Successful!" & vbCrLf &"Registration
               Date: " & dtpRegister.Value.ToLongDateString,"Success",
                      MessageBoxButtons.OK, MessageBoxIcon.Information)
               ClearFields()
               Login.Show()
               Me.Close()
        Catch ex As Exception
               MessageBox.Show(ex.Message)
        Finally
        con.Close()
        End Try
        End Sub

    Private Sub ClearFields()
        txtUser.Clear()
        txtPassword.Clear()
        txtConfirmPassword.Clear()
        cmbCourse.Text = ""
        txtEmail.Clear()
        txtMobile.Clear()
    End Sub

    Private Sub Register_Load(sender As Object, e As EventArgs) Handles MyBase.Load
        Me.Width = 900
        Me.Height = 700

        cmbCourse.Items.Clear()
        cmbCourse.Items.AddRange(New String() {"Computer Science", "Commerce",
                             "Science", "Arts", "Digital Marketing", "Law"})
        cmbCourse.SelectedIndex = 0
        cmbCourse.Text = "  Select a Course  "
    End Sub

  End Class
```

# 3. Membership :

```
Imports System.Data.OleDb
Public Class Membership
ReadOnly conString As String = "Provider = Microsoft.ACE.OLEDB.12.0; Data Source =
     F:\Project (Library Management)\My Project (LibraryManagement)\ Project\ bin
                  \ Debug\LibraryManagementDB.accdb;Persist Security Info=False;"
Dim regConString As String = "Provider = Microsoft.ACE.OLEDB.12.0; Data Source =
     F:\Project (Library Management)\My Project (Library Management)\ Project\ bin
                                                 \Debug\LibraryManagementDB.accdb"

Private Sub Membership_Load(sender As Object, e As EventArgs) Handles MyBase.Load
 dtpRegister.Value = DateTime.Now
 dtpDOB.Format = DateTimePickerFormat.Short
 dtpRegister.Format = DateTimePickerFormat.Short
 cmbPlan.Items.Clear()
 cmbPlan.Items.AddRange(New String() {"Monthly", "Quarterly", "Half-Yearly",
                                                          "Yearly"})
 cmbPlan.SelectedIndex = -1
 cmbGender.Items.Clear()
 cmbGender.Items.AddRange(New String() {"Male", "Female"})
 cmbGender.SelectedIndex = -1
 cmbType.Items.Clear()
 cmbType.Items.AddRange(New String() {"Student", "Teacher", "General-Public"})
 cmbType.SelectedIndex = -1
 cmbPmethod.Items.AddRange(New String() {"Cash", "UPI"})
 cmbPmethod.SelectedIndex = -1
 CalculateFeesAndExpiry()
End Sub

Private Sub cmbPlan_SelectedIndexChanged_1(sender As Object, e As EventArgs)
                                       Handles cmbPlan.SelectedIndexChanged
       CalculateFeesAndExpiry()
End Sub
Private Sub dtpRegister_ValueChanged_1(sender As Object, e As EventArgs) Handles
                                                        dtpRegister.ValueChanged
       CalculateFeesAndExpiry()
End Sub
Private Sub CalculateFeesAndExpiry()
      Dim plan As String = If(cmbPlan.SelectedItem IsNot Nothing,
                              cmbPlan.SelectedItem.ToString(), "")
       Dim subDate As DateTime = dtpRegister.Value.Date
       Dim expiry As DateTime = subDate
       Dim feesAmount As Decimal = 0D

       Select Case plan
              Case "Monthly"
              expiry = subDate.AddMonths(1).AddDays(-1)
              feesAmount = 300
              Case "Quarterly"
              expiry = subDate.AddMonths(3).AddDays(-1)
              feesAmount = 800
              Case "Half-Yearly"
              expiry = subDate.AddYears(1).AddDays(-1)
              feesAmount = 1500
              Case "Yearly"
              expiry = subDate.AddYears(1).AddDays(-1)
              feesAmount = 3000
              Case Else
```

```
            expiry = subDate
            feesAmount = 0
        End Select
        txtExpiry.Text = expiry.ToShortDateString()
        txtFees.Text = feesAmount.ToString("F2")
    End Sub

    Private Sub btnSave_Click(sender As Object, e As EventArgs) Handles btnSave.Click
        If String.IsNullOrWhiteSpace(txtName.Text) Then
          MessageBox.Show("Please enter member name.", "Validation",
                          MessageBoxButtons.OK, MessageBoxIcon.Warning)
          txtName.Focus()
        Return
        End If
        If cmbPmethod.Text = "" Then
          MessageBox.Show("Please select a payment method before purchasing!",
                      "Warning", MessageBoxButtons.OK, MessageBoxIcon.Warning)
        Exit Sub
        End If
        Dim idVal As String = txtID.Text.Trim()
        Dim nameVal As String = txtName.Text.Trim()
        Dim dobVal As Date = dtpDOB.Value.Date
        Dim genderVal As String = If(cmbGender.SelectedItem IsNot Nothing,
                                     cmbGender.SelectedItem.ToString(), "")
        Dim emailVal As String = txtEmail.Text.Trim()
        Dim mobileVal As String = txtMobile.Text.Trim()
        Dim addressVal As String = txtAddress.Text.Trim()
        Dim typeVal As String = If(cmbType.SelectedItem IsNot Nothing,
                                     cmbType.SelectedItem.ToString(), "")
        Dim planVal As String = If(cmbPlan.SelectedItem IsNot Nothing,
                                     cmbPlan.SelectedItem.ToString(), "")
        Dim subDateVal As Date = dtpRegister.Value.Date
        Dim payVal As String = If(cmbPmethod.SelectedItem IsNot Nothing,
                                     cmbPmethod.SelectedItem.ToString(), "")

        Dim feesVal As Decimal = 0
        Decimal.TryParse(txtFees.Text, feesVal)
        Dim expiryVal As Date = DateTime.Now
        Date.TryParse(txtExpiry.Text, expiryVal)

    Using conCheck As New OleDbConnection(conString)
        Dim checkSql As String = "SELECT * FROM Membership WHERE MemberID = ? AND
                                                                 ExpiryDate >= ?"

         Using cmdCheck As New OleDbCommand(checkSql, conCheck)
            cmdCheck.Parameters.AddWithValue("?", idVal)
            cmdCheck.Parameters.AddWithValue("?", Date.Today)

            conCheck.Open()

            Dim rdr As OleDbDataReader = cmdCheck.ExecuteReader()

            If rdr.HasRows Then
                MessageBox.Show("You already have an active membership. You can take
                        another membership only after this one expires.","Membership
                        Exists",MessageBoxButtons.OK,MessageBoxIcon.Warning)
                Exit Sub
            End If
        End Using
    End Using
```

```
        Using con As New OleDbConnection(conString)
            Dim sql As String = "INSERT INTO Membership ([MemberID], [Name], [DOB],
              [Gender], [Email], [Mobile], [Address], [MembershipType], [MembershipPlan],
              [RegistrationDate], [ExpiryDate], [Fees], [PaymentMethod]) " &
                                 "VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?, ?)"
            Using cmd As New OleDbCommand(sql, con)
                cmd.Parameters.AddWithValue("?", idVal)
                cmd.Parameters.AddWithValue("?", nameVal)
                cmd.Parameters.AddWithValue("?", dobVal)
                cmd.Parameters.AddWithValue("?", genderVal)
                cmd.Parameters.AddWithValue("?", emailVal)
                cmd.Parameters.AddWithValue("?", mobileVal)
                cmd.Parameters.AddWithValue("?", addressVal)
                cmd.Parameters.AddWithValue("?", typeVal)
                cmd.Parameters.AddWithValue("?", planVal)
                cmd.Parameters.AddWithValue("?", subDateVal)
                cmd.Parameters.AddWithValue("?", expiryVal)
                cmd.Parameters.AddWithValue("?", feesVal)
                cmd.Parameters.AddWithValue("?", payVal)
                Try
                    con.Open()
                    cmd.ExecuteNonQuery()
                    MessageBox.Show("Member Registered Successful!", "Success",
                     MessageBoxButtons.OK, MessageBoxIcon.Information)
                Catch ex As Exception
                    MessageBox.Show("Error while saving: " & ex.Message, "Error",
                     MessageBoxButtons.OK, MessageBoxIcon.Error)
                End Try
            End Using
        End Using
        UserInterface.Show()
        Me.Close()
    End Sub

    Private Sub btnClear_Click(sender As Object, e As EventArgs) Handles btnClear.Click
        ClearInputs(excludeId:=False)
    End Sub

    Private Sub ClearInputs(excludeId As Boolean)
        If Not excludeId Then txtID.Text = ""
        txtName.Text = ""
        dtpDOB.Value = DateTime.Now.AddYears(-20)
        cmbGender.SelectedIndex = 0
        txtEmail.Text = ""
        txtMobile.Text = ""
        txtAddress.Text = ""
        cmbType.SelectedIndex = -1
        cmbPlan.SelectedIndex = 0
        dtpRegister.Value = DateTime.Now
        cmbPmethod.SelectedIndex = 0
        CalculateFeesAndExpiry()
    End Sub

    Private Sub PictureBox1_Click_1(sender As Object, e As EventArgs) Handles
                                                                PictureBox1.Click
        UserInterface.Show()
        Me.Close()
    End Sub
```

```
Private Sub txtName_LostFocus(sender As Object, e As EventArgs) Handles
                                                    txtName.LostFocus
    AutoFillMemberData()
End Sub
Private Sub txtName_KeyDown(sender As Object, e As KeyEventArgs) Handles
                                                     txtName.KeyDown
    If e.KeyCode = Keys.Enter Then
        AutoFillMemberData()
    End If
End Sub

Private Sub AutoFillMemberData()
    If txtName.Text.Trim() = "" Then Exit Sub

    Using con As New OleDbConnection(regConString)
        Dim query As String = "SELECT * FROM Register WHERE Username = @u"

        Using cmd As New OleDbCommand(query, con)
            cmd.Parameters.AddWithValue("@u", txtName.Text.Trim())
            Try
                con.Open()
                Dim reader As OleDbDataReader = cmd.ExecuteReader()

                If reader.Read() Then
                    txtEmail.Text = reader("EMail").ToString()
                    txtMobile.Text = reader("MobileNo").ToString()
                    txtID.Text = reader("MemberID").ToString()
                    dtpRegister.Value = Convert.ToDateTime
                                    (reader("RegistrationDate"))
                Else
                    MessageBox.Show("Name Not Found In Registration!")
                End If

            Catch ex As Exception
                MessageBox.Show("Error loading registration data: " & ex.Message)
            End Try

        End Using
    End Using
End Sub

Private Sub cmbPmethod_SelectedIndexChanged(sender As Object, e As EventArgs) Handles
                                                    cmbPmethod.SelectedIndexChanged
    If cmbPmethod.Text = "UPI" Then
        picMQR.Visible = True
    Else
        picMQR.Visible = False
    End If
End Sub
End Class
```

# 4. Book Details :

```
Imports System.Data.OleDb
Public Class UserBookDetail
       Dim con As New OleDbConnection("Provider = Microsoft.ACE.OLEDB.12.0; Data
       Source = F:\Project (Library Management)\My Project (Library Management)\
       Project\bin\Debug\LibraryManagementDB.accdb; Persist Security Info=False;")
Public SelectedBookID As Integer
```

```
Private Sub PictureBox1_Click(sender As Object, e As EventArgs) Handles
                                                PictureBox1.Click
 UserInterface.Show()
 Me.Close()
End Sub
Private Sub UserBookDetail_Load(sender As Object, e As EventArgs) Handles
                                                        MyBase.Load
 LoadAllBooks()
 PanelBooks.AutoScroll = True
 PanelBooks.AutoScrollMargin = New Size(0, 20)
 PanelBooks.Padding = New Padding(0, 0, 10, 0)
 If SelectedBookID > 0 Then
     LoadBookDetails(SelectedBookID)
 End If
End Sub
Private Sub LoadAllBooks()
 PanelBooks.Controls.Clear()
 Dim cmd As New OleDbCommand("SELECT * FROM BookDetail ORDER BY BookID", con)
 con.Open()
 Dim dr As OleDbDataReader = cmd.ExecuteReader()
 Dim yPos As Integer = 10
 While dr.Read()
     GlobalVariables.SelectedBookTitle = lblTitle.Text
     GlobalVariables.SelectedBookAuthor = lblAuthor.Text
     Dim bookPanel As New Panel With {
         .Width = 450,
         .Height = 135,
         .BorderStyle = BorderStyle.FixedSingle,
         .Location = New Point(10, yPos),
         .BackColor = Color.White}
     Dim pic As New PictureBox With {
         .Width = 71,
         .Height = 106,
         .SizeMode = PictureBoxSizeMode.StretchImage,
         .Location = New Point(30, 10)}
     Try
         pic.Image =
       My.Resources.ResourceManager.GetObject(dr("ResourceName").ToString())
     Catch
         pic.Image = Nothing
     End Try
     Dim lbl As New Label With {
         .Text = dr("Title").ToString(),
         .Font = New Font("Times New Roman", 14, FontStyle.Bold),
         .ForeColor = Color.Navy,
         .AutoSize = True,
         .Location = New Point(125, 22)}
     Dim btnDetail As New Button With
     {
         .Text = "Detail",
         .BackColor = Color.LightGreen,
         .Font = New Font("Times New Roman", 14, FontStyle.Bold),
         .Width = 80,
         .Height = 35,
         .Cursor = Cursors.Hand,
         .Location = New Point(140, 70)}
     btnDetail.Tag = dr("BookID")
     AddHandler btnDetail.Click, AddressOf btnDetail_Click
     bookPanel.Controls.Add(pic)
     bookPanel.Controls.Add(lbl)
```

```
            bookPanel.Controls.Add(btnDetail)
            PanelBooks.Controls.Add(bookPanel)
            yPos += 140
        End While
        dr.Close()
        con.Close()
    End Sub
    Private Sub LoadBookDetails(bookID As Integer)
        Try
            Dim cmd As New OleDbCommand("SELECT * FROM BookDetail WHERE BookID=@id", con)
            cmd.Parameters.AddWithValue("@id", bookID)
            con.Open()
            Dim dr As OleDbDataReader = cmd.ExecuteReader()
            If dr.Read() Then
                lblBookID.Text = dr("BookID").ToString()
                lblTitle.Text = dr("Title").ToString()
                lblAuthor.Text = dr("Author").ToString()
                lblPublisher.Text = dr("Publisher").ToString()
                lblYear.Text = dr("BookYear").ToString
                lblAvailability.Text = dr("AvailableCopies").ToString
                lblPrice.Text = dr("Price").ToString()
                btnIssue.Tag = dr("BookID")
                btnPurchase.Tag = dr("BookID")
                Try
                    pbDetail.Image =
                 My.Resources.ResourceManager.GetObject(dr("ResourceName").ToString())
                Catch
                    pbDetail.Image = Nothing
                End Try
            End If
            dr.Close()
            con.Close()
               Catch ex As Exception
                     MessageBox.Show("Error loading book: " & ex.Message)
              End Try
    End Sub
    Private Sub btnDetail_Click(sender As Object, e As EventArgs) Handles btnDetail.Click
        Dim bookID As Integer = CType(sender, Button).Tag
        Dim cmd As New OleDbCommand("SELECT * FROM BookDetail WHERE BookID=" & bookID,
                                                                                    con)
        con.Open()
        Dim dr As OleDbDataReader = cmd.ExecuteReader()
        If dr.Read() Then
            lblBookID.Text = dr("BookID").ToString()
            lblTitle.Text = dr("Title").ToString()
            lblAuthor.Text = dr("Author").ToString()
            lblPublisher.Text = dr("Publisher").ToString()
            lblYear.Text = dr("BookYear").ToString()
            lblAvailability.Text = dr("AvailableCopies").ToString()
            lblPrice.Text = dr("Price").ToString()
            btnIssue.Tag = dr("BookID")
            btnPurchase.Tag = dr("BookID")
            Try
                pbDetail.Image =
               My.Resources.ResourceManager.GetObject(dr("ResourceName").ToString())
            Catch
                pbDetail.Image = Nothing
            End Try
        End If
```

```
        dr.Close()
        con.Close()
    End Sub
    Private Sub btnSearch_Click(sender As Object, e As EventArgs) Handles btnSearch.Click
        Dim keyword As String = txtSearch.Text.Trim()

        If keyword = "" Then
            LoadAllBooks()
        Else
            LoadSearchedBooks(keyword)
        End If
    End Sub
    Private Sub LoadSearchedBooks(keyword As String)
        PanelBooks.Controls.Clear()
       Dim cmd As New OleDbCommand("SELECT * FROM BookDetail WHERE Title LIKE @key ORDER
                                                                    BY BookID", con)
        cmd.Parameters.AddWithValue("@key", "%" & keyword & "%")
        con.Open()
        Dim dr As OleDbDataReader = cmd.ExecuteReader()
        Dim yPos As Integer = 10
        While dr.Read()
            Dim bookPanel As New Panel With {
                .Width = 450,
                .Height = 135,
                .BorderStyle = BorderStyle.FixedSingle,
                .Location = New Point(10, yPos),
                .BackColor = Color.White}
            Dim pic As New PictureBox With {
                .Width = 71,
                .Height = 106,
                .SizeMode = PictureBoxSizeMode.StretchImage,
                .Location = New Point(30, 10)}
            Try
                pic.Image =
               My.Resources.ResourceManager.GetObject(dr("ResourceName").ToString())
            Catch
                pic.Image = Nothing
            End Try
            Dim lbl As New Label With {
                .Text = dr("Title").ToString(),
                .Font = New Font("Times New Roman", 14, FontStyle.Bold),
                .ForeColor = Color.Navy,
                .AutoSize = True,
                .Location = New Point(125, 22)}
            Dim btnDetail As New Button With {
                .Text = "Detail",
                .BackColor = Color.LightGreen,
                .Font = New Font("Times New Roman", 14, FontStyle.Bold),
                .Width = 80,
                .Height = 35,
                .Location = New Point(160, 70)}
            btnDetail.Tag = dr("BookID")
            AddHandler btnDetail.Click, AddressOf btnDetail_Click
            bookPanel.Controls.Add(pic)
            bookPanel.Controls.Add(lbl)
            bookPanel.Controls.Add(btnDetail)
            PanelBooks.Controls.Add(bookPanel)
            yPos += 140
        End While
```

```
        dr.Close()
        con.Close()
    End Sub
    Private Function AreBookDetailsComplete() As Boolean
        If lblBookID.Text = "" Then Return False
        If lblTitle.Text = "" Then Return False
        If lblAuthor.Text = "" Then Return False
        If lblPublisher.Text = "" Then Return False
        If lblYear.Text = "" Then Return False
        If lblAvailability.Text = "" Then Return False
        If lblPrice.Text = "" Then Return False
        Return True
    End Function
    Private Sub btnPurchase_Click(sender As Object, e As EventArgs) Handles
                                                                  btnPurchase.Click
        If Not AreBookDetailsComplete() Then
            MessageBox.Show("Book details are not fully loaded. Please select a book
                                                                          first.")
            Exit Sub
        End If
        Dim bookID As Integer = CInt(CType(sender, Button).Tag)
        Try
            con.Open()
            Dim cmd As New OleDbCommand("UPDATE BookDetail SET AvailableCopies =
              AvailableCopies - 1 WHERE BookID = @id AND AvailableCopies > 0", con)
            cmd.Parameters.AddWithValue("@id", bookID)
            Dim rows As Integer = cmd.ExecuteNonQuery()
            If rows > 0 Then
                MessageBox.Show("Book added successfully! Redirecting you to the payment
       section.", "Proceed to Payment", MessageBoxButtons.OK, MessageBoxIcon.Information)
            Else
                MessageBox.Show("Book is not available!")
            End If
        Catch ex As Exception
            MessageBox.Show(ex.Message)
        Finally
            con.Close()
        End Try
        LoadBookDetails(bookID)
        Dim pf As New PaymentPurchaseBook()
        pf.UserName = LoggedUserName
        pf.BookTitle = lblTitle.Text
        pf.BookAuthor = lblAuthor.Text
        pf.BookPrice = lblPrice.Text
        pf.BookImage = pbDetail.Image
        pf.selectedBookID = SelectedBookID
        pf.Show()
        Me.Close()
    End Sub
    Private Sub btnIssue_Click(sender As Object, e As EventArgs) Handles btnIssue.Click
        If Not AreBookDetailsComplete() Then
            MessageBox.Show("Book details are not fully loaded. Please select a book
                                                                          first.")
            Exit Sub
        End If
        If Not IsMembershipValid(User.LoggedUserName) Then
            MessageBox.Show("You don't have an active membership! Please get the
                     membership for issue and renew books.", "Membership Required",
                                  MessageBoxButtons.OK, MessageBoxIcon.Warning)
            Membership.Show()
```

```
            Me.Close()
            Exit Sub
        End If
        Dim bookID As Integer = CInt(CType(sender, Button).Tag)
        Try
            con.Open()
            Dim cmd As New OleDbCommand("UPDATE BookDetail SET AvailableCopies =
              AvailableCopies - 1 WHERE BookID = @id AND AvailableCopies > 0", con)
            cmd.Parameters.AddWithValue("@id", bookID)
            Dim rows As Integer = cmd.ExecuteNonQuery()
            If rows > 0 Then
                MessageBox.Show("Book added successfully! Redirecting you to the Issue
                                                                Book section")
            Else
                MessageBox.Show("Book is not available!")
            End If
        Catch ex As Exception
            MessageBox.Show(ex.Message)
        Finally
            con.Close()
        End Try
        LoadBookDetails(bookID)
        Dim pf As New PaymentIssueBook()
        pf.UserName = LoggedUserName
        pf.BookTitle = lblTitle.Text
        pf.BookAuthor = lblAuthor.Text
        pf.BookImage = pbDetail.Image
        pf.selectedBookID = SelectedBookID
        pf.Show()
        Me.Close()
    End Sub
    Private Function IsMembershipValid(userName As String) As Boolean
        Try
            Dim cmd As New OleDbCommand("SELECT ExpiryDate FROM Membership WHERE Name =
                                                                        @name", con)
            cmd.Parameters.AddWithValue("@name", userName)
            con.Open()
            Dim dr As OleDbDataReader = cmd.ExecuteReader()
            If dr.Read() Then
                Dim expiry As Date = Convert.ToDateTime(dr("ExpiryDate"))
                dr.Close()
                con.Close()
                If expiry >= Date.Today Then
                    Return True
                Else
                    Return False
                End If
            Else
                dr.Close()
                con.Close()
                Return False
            End If
       Catch ex As Exception
            MessageBox.Show("Error checking membership: " & ex.Message)
            con.Close()
            Return False
        End Try
    End Function
End Class
```

# 5. Issue Book :

```
Imports System.Data.OleDb
Public Class PaymentIssueBook
       Dim con As New OleDbConnection("Provider = Microsoft.ACE.OLEDB.12.0; Data
       Source = F:\Project (Library Management)\My Project (Library Management)\
       Project\bin\Debug\LibraryManagementDB.accdb; Persist Security Info=False;")
Public Property UserName As String
Public Property BookTitle As String
Public Property BookAuthor As String
Public Property BookImage As Image
Public selectedBookID As Integer
Public FromMembership As Boolean = False
Public FromBookDetail As Boolean = False
Private Sub PaymentInterface1_Load(sender As Object, e As EventArgs) Handles
                                                                  MyBase.Load
 btnPrint.Enabled = False
 PrintPreviewDialog1.Document = PrintDocument1
 btnCancel.Enabled = True
 picBook.Image = BookImage
 txtName.Text = User.LoggedUserName
 txtTitle.Text = BookTitle
 txtAuthor.Text = BookAuthor
 lblStatus.Text = "Issue"
 Try
     Using con As New OleDbConnection("Provider = Microsoft.ACE.OLEDB.12.0; Data
       Source = F:\Project (Library Management)\My Project (Library Management)\
     Project\bin\Debug\LibraryManagementDB.accdb; Persist Security Info=False;")
         Dim query As String = "SELECT MembershipType, MembershipPlan FROM
                                          Membership WHERE Name = @u"
         Using cmd As New OleDbCommand(query, con)
             cmd.Parameters.AddWithValue("@u", txtName.Text.Trim())
             con.Open()
             Dim dr As OleDbDataReader = cmd.ExecuteReader()
             If dr.Read() Then
                 txtType.Text = dr("MembershipType").ToString()
                 txtPlan.Text = dr("MembershipPlan").ToString()
             Else
                 MessageBox.Show("Membership details not found for this user!",
                                                                  "Warning")
             End If
         End Using
     End Using
 Catch ex As Exception
     MessageBox.Show("Error loading membership details: " & ex.Message)
 End Try
End Sub
Private Function GenerateInvoiceNo() As String
    Dim newInvoice As String = ""
    Dim yearPart As String = DateTime.Now.Year.ToString()
    Dim lastNumber As Integer = 0
    Try
        Dim cmd As New OleDbCommand("SELECT InvoiceNo FROM IssueBook WHERE InvoiceNo
                 LIKE 'INV-" & yearPart & "-%' ORDER BY InvoiceNo DESC", con)

        con.Open()
        Dim dr As OleDbDataReader = cmd.ExecuteReader()
        If dr.Read() Then
            Dim lastInv As String = dr("InvoiceNo").ToString()
```

```
            Dim numberPart As String = lastInv.Substring(lastInv.LastIndexOf("-") + 1)
             lastNumber = CInt(numberPart)
        End If
        con.Close()
        lastNumber += 1
        newInvoice = "INV-" & yearPart & "-" & lastNumber.ToString("000")
    Catch ex As Exception
        MessageBox.Show(ex.Message)
        con.Close()
    End Try
    Return newInvoice
End Function
Private Sub btnConfirm_Click(sender As Object, e As EventArgs) Handles
                                                    btnConfirm.Click
    Try
        Dim checkCmd As New OleDbCommand("SELECT COUNT(*) FROM IssueBook WHERE
        UserName = @UserName AND BookTitle = @BookTitle  AND Status = 'Issue'", con)
        checkCmd.Parameters.AddWithValue("@UserName", txtName.Text)
        checkCmd.Parameters.AddWithValue("@BookTitle", txtTitle.Text)
        con.Open()
        Dim alreadyIssued As Integer = Convert.ToInt32(checkCmd.ExecuteScalar())
        con.Close()
        If alreadyIssued > 0 Then
            MessageBox.Show("You have already issued this book.","Already
                        Issued",MessageBoxButtons.OK,MessageBoxIcon.Warning)
            Exit Sub
        End If
        Dim invoiceNo As String = GenerateInvoiceNo()
        lblInvoiceNumber.Text = invoiceNo
        con.Open()
        Dim cmd As New OleDbCommand("INSERT INTO IssueBook (InvoiceNo, UserName,
          BookTitle, BookAuthor, IssueDate, UserType, UserMembership, Status) VALUES
          (@InvoiceNo, @UserName, @BookTitle, @BookAuthor, @IssueDate, @UserType,
                                          @UserMembership, @Status)", con)
        cmd.Parameters.AddWithValue("@InvoiceNo", lblInvoiceNumber.Text)
        cmd.Parameters.AddWithValue("@UserName", txtName.Text)
        cmd.Parameters.AddWithValue("@BookTitle", txtTitle.Text)
        cmd.Parameters.AddWithValue("@BookAuthor", txtAuthor.Text)
        cmd.Parameters.AddWithValue("@IssueDate", dtpIssueDate.Value.Date)
        cmd.Parameters.AddWithValue("@UserType", txtType.Text)
        cmd.Parameters.AddWithValue("@UserMembership", txtPlan.Text)
        cmd.Parameters.AddWithValue("@Status", lblStatus.Text)
        cmd.ExecuteNonQuery()
        MessageBox.Show("Book Issued Successfully!", "Success")
        btnPrint.Enabled = True
        con.Close()
    Catch ex As Exception
        MessageBox.Show("Error: " & ex.Message)
    End Try
End Sub
Private Sub btnCancel_Click(sender As Object, e As EventArgs) Handles btnCancel.Click
    Try
        Dim query As String = "UPDATE BookDetail SET AvailableCopies =
          AvailableCopies + 1 WHERE BookID = ?"
        Using cmd As New OleDbCommand(query, con)
            cmd.Parameters.AddWithValue("?", selectedBookID)
            con.Open()
            cmd.ExecuteNonQuery()
            con.Close()
        End Using
```

```
            MessageBox.Show("Issued cancelled. Book stock restored.", "Cancelled",
                                    MessageBoxButtons.OK, MessageBoxIcon.Information)
        Catch ex As Exception
            MessageBox.Show(ex.Message)
            con.Close()
        End Try
    End Sub
    Private Sub btnPrint_Click(sender As Object, e As EventArgs) Handles btnPrint.Click
        PrintPreviewDialog1.Width = 900
        PrintPreviewDialog1.Height = 600
        PrintPreviewDialog1.StartPosition = FormStartPosition.CenterScreen
        PrintPreviewDialog1.WindowState = FormWindowState.Maximized
        PrintPreviewDialog1.ShowDialog()
        btnCancel.Enabled = False
    End Sub
    Private Sub PrintDocument1_PrintPage(sender As Object, e A Printing.
                    PrintPageEventArgs)Handles PrintDocument1.PrintPage
        Dim g As Graphics = e.Graphics
        Dim titleFont As New Font("Times New Roman", 18, FontStyle.Bold)
        Dim headFont As New Font("Times New Roman", 14, FontStyle.Bold)
        Dim bodyFont As New Font("Times New Roman", 12)
        Dim x As Integer = 50
        Dim y As Integer = 40
        g.DrawString("LIBRARY ISSUE BOOK INVOICE", titleFont, Brushes.Black, x + 150, y)
        y += 40
        y += 40
        g.DrawString("Invoice No : " &lblInvoiceNumber.Text,bodyFont,Brushes.Black, x, y)
        y += 20
        g.DrawString("Issue Date : " & dtpIssueDate.Value.ToString("dd-MMM-yyyy"),
                                                bodyFont, Brushes.Black, x, y)
        y += 30
        g.DrawLine(Pens.Black, x, y, 750, y)
        y += 20
        g.DrawString("User Information", headFont, Brushes.Black, x, y)
        y += 25
        g.DrawString("Name : " & txtName.Text, bodyFont, Brushes.Black, x, y)
        y += 20
        g.DrawString("User Type : " & txtType.Text, bodyFont, Brushes.Black, x, y)
        y += 20
        g.DrawString("Membership Plan : " & txtPlan.Text, bodyFont, Brushes.Black, x, y)
        y += 25
        g.DrawLine(Pens.Black, x, y, 750, y)
        y += 20
        g.DrawString("Book Details", headFont, Brushes.Black, x, y)
        y += 25
        g.DrawString("Title : " & txtTitle.Text, bodyFont, Brushes.Black, x, y)
        y += 20
        g.DrawString("Author : " & txtAuthor.Text, bodyFont, Brushes.Black, x, y)
        y += 30
        g.DrawLine(Pens.Black, x, y, 750, y)
        y += 30
        g.DrawString("Issued Successfully", headFont, Brushes.Black, x + 263, y)
        y += 25
        g.DrawString("Please return the book on time.",bodyFont,Brushes.Black,x + 240, y)
    End Sub
End Class
```

# 6. Overdue and Fine :

```
Imports System.Data.OleDb
Public Class OverdueFineForm
       Dim con As New OleDbConnection("Provider = Microsoft.ACE.OLEDB.12.0; Data
       Source = F:\Project (Library Management)\My Project (Library Management)\
       Project\bin\Debug\LibraryManagementDB.accdb; Persist Security Info=False;")
       Dim lastFinePaidDate As Date = Date.MinValue
Private Sub PictureBox1_Click(sender As Object, e As EventArgs) Handles
                                                        PictureBox1.Click
    UserInterface.Show()
    Me.Close()
End Sub
Private Sub DeleteExpiredMemberships()

 Dim query As String = "DELETE FROM Membership WHERE ExpiryDate < @today"
 Dim cmd As New OleDbCommand(query, con)
 cmd.Parameters.AddWithValue("@today", Date.Today)
 con.Open()
 Dim rows As Integer = cmd.ExecuteNonQuery()
 con.Close()
 If rows > 0 Then
     MessageBox.Show("Expired membership data removed automatically!")
 End If
End Sub
Private Sub OverdueFineForm_Load(sender As Object, e As EventArgs) Handles
                                                              MyBase.Load
 LoadMembership()
 LoadPurchasedBooks()
 LoadIssuedBooks()
 LoadOverdueAndFine()
 DeleteExpiredMemberships()
 GetLastInvoiceNo()
 btnPay.Enabled = False
 btnCancel.Enabled = False
 btnPrint.Enabled = False
 picQR.Visible = False
End Sub
Private Sub LoadMembership()
    Dim query As String = "SELECT * FROM Membership WHERE Name = @user"
    Dim cmd As New OleDbCommand(query, con)
    cmd.Parameters.AddWithValue("@user", User.LoggedUserName)
    Dim da As New OleDbDataAdapter(cmd)
    Dim dt As New DataTable()
    da.Fill(dt)
    dgvMembership.DataSource = dt
End Sub
Private Sub LoadPurchasedBooks()
    Dim query As String = "SELECT * FROM PurchaseBook WHERE UserName = @user"
    Dim cmd As New OleDbCommand(query, con)
    cmd.Parameters.AddWithValue("@user", User.LoggedUserName)
    Dim da As New OleDbDataAdapter(cmd)
    Dim dt As New DataTable()
    da.Fill(dt)
    dgvPurchase.DataSource = dt
End Sub
Private Sub LoadIssuedBooks()
    Dim query As String = "SELECT * FROM IssueBook WHERE UserName = @user"
    Dim cmd As New OleDbCommand(query, con)
```

```
        cmd.Parameters.AddWithValue("@user", User.LoggedUserName)
        Dim da As New OleDbDataAdapter(cmd)
        Dim dt As New DataTable()
        da.Fill(dt)
        dgvIssue.DataSource = dt
    End Sub
    Private Sub LoadOverdueAndFine()
        Dim totalOverdueDays As Integer = 0
        Dim finePerDay As Integer = 10
        If dgvIssue.Rows.Count = 0 Or dgvIssue.Rows(0).IsNewRow Then
            rbCash.Enabled = False
            rbUpi.Enabled = False
            btnPay.Enabled = False
            btnCancel.Enabled = False
            picQR.Visible = False
            Exit Sub
        End If
        For Each row As DataGridViewRow In dgvIssue.Rows
            If row.IsNewRow Then Continue For
            Dim issueDate As Date = Convert.ToDateTime(row.Cells("IssueDate").Value)
            Dim daysPassed As Integer = (Date.Today - issueDate).Days
            If daysPassed > 14 Then
                totalOverdueDays += (daysPassed - 14)
            End If
        Next
        txtOverdueDays.Text = totalOverdueDays.ToString()
        txtFine.Text = "Rs " & (totalOverdueDays * finePerDay).ToString()
        If totalOverdueDays > 0 Then
            rbCash.Enabled = True
            rbUpi.Enabled = True
        Else
            rbCash.Enabled = False
            rbUpi.Enabled = False
            btnPay.Enabled = False
            btnCancel.Enabled = False
            picQR.Visible = False
        End If
    End Sub
    Private Sub rbCash_CheckedChanged(sender As Object, e As EventArgs) Handles
                                                        rbCash.CheckedChanged
        If rbCash.Checked = True Then
            btnPay.Enabled = True
            btnCancel.Enabled = True
            picQR.Visible = False
        End If
    End Sub
    Private Sub rbUpi_CheckedChanged(sender As Object, e As EventArgs) Handles
                                                        rbUpi.CheckedChanged
        If rbUpi.Checked = True Then
            btnPay.Enabled = True
            btnCancel.Enabled = True
            picQR.Visible = True
        End If
    End Sub
    Private Sub btnPay_Click(sender As Object, e As EventArgs) Handles btnPay.Click
        If rbCash.Checked = False AndAlso rbUpi.Checked = False Then
            MessageBox.Show("Select payment method!")
            Exit Sub
        End If
        Dim paymentMethod As String = If(rbCash.Checked, "Cash", "UPI")
```

```
        Dim overdueDays As Integer = Val(txtOverdueDays.Text)
        Dim fineAmount As Integer = overdueDays * 10
        Dim query As String = "INSERT INTO [Overdue&Fine](InvoiceNo, Username,
              OverdueDays, FineAmount, PaymentMethod, PaidDate) VALUES (@InvoiceNo,
                                          @user, @days, @amount, @method, @date)"
        Dim cmd As New OleDbCommand(query, con)
        cmd.Parameters.AddWithValue("@InvoiceNo", GetLastInvoiceNo.ToString())
        cmd.Parameters.AddWithValue("@user", User.LoggedUserName)
        cmd.Parameters.AddWithValue("@days", overdueDays.ToString())
        cmd.Parameters.AddWithValue("@amount", fineAmount.ToString())
        cmd.Parameters.AddWithValue("@method", paymentMethod)
        cmd.Parameters.AddWithValue("@date", Date.Today)
        con.Open()
        cmd.ExecuteNonQuery()
        con.Close()
        rbCash.Checked = False
        rbUpi.Checked = False
        btnPay.Enabled = False
        btnCancel.Enabled = False
        picQR.Visible = False
        MessageBox.Show("Fine Paid Successfully & Issue Date Reset!", "Success")
        btnPrint.Enabled = True
    End Sub
    Private Sub btnCancel_Click(sender As Object, e As EventArgs) Handles btnCancel.Click
        rbCash.Checked = False
        rbUpi.Checked = False
        picQR.Visible = False
        btnPay.Enabled = False
        btnCancel.Enabled = False
        MessageBox.Show("Payment has been cancelled.", "Cancelled")
    End Sub
    Private Function GetLastInvoiceNo() As String
        Dim newInvoice As String = ""
        Dim yearPart As String = DateTime.Now.Year.ToString()
        Dim lastNumber As Integer = 0
        Try
            Dim cmd As New OleDbCommand("SELECT InvoiceNo FROM [Overdue&Fine] WHERE
              InvoiceNo LIKE 'INV-" & yearPart & "-%' ORDER BY InvoiceNo DESC", con)
            con.Open()
            Dim dr As OleDbDataReader = cmd.ExecuteReader()
            If dr.Read() Then
               Dim lastInv As String = dr("InvoiceNo").ToString()
               Dim numberPart As String = lastInv.Substring(lastInv.LastIndexOf("-") + 1)
               lastNumber = CInt(numberPart)
            End If
            con.Close()
            lastNumber += 1
            newInvoice = "INV-" & yearPart & "-" & lastNumber.ToString("000")
        Catch ex As Exception
            MessageBox.Show(ex.Message)
            con.Close()
        End Try
        Return newInvoice
    End Function
    Private Sub ResetIssueDateAfterFine()
        Dim query As String = "UPDATE IssueBook SET IssueDate = @today WHERE UserName =
                                  @user AND DateDiff('d', IssueDate, @today) > 14"
        Dim cmd As New OleDbCommand(query, con)
        cmd.Parameters.AddWithValue("@today", Date.Today)
        cmd.Parameters.AddWithValue("@user", User.LoggedUserName)
```

```
        con.Open()
        cmd.ExecuteNonQuery()
        con.Close()
    End Sub
    Private Sub btnPrint_Click(sender As Object, e As EventArgs) Handles btnPrint.Click
        PrintPreviewDialog1.Document = PrintDocument1
        PrintPreviewDialog1.StartPosition = FormStartPosition.CenterScreen
        PrintPreviewDialog1.WindowState = FormWindowState.Maximized
        PrintPreviewDialog1.ShowDialog()
        txtOverdueDays.Text = "0"
        txtFine.Text = "Rs 0"
        ResetIssueDateAfterFine()
        LoadIssuedBooks()
        LoadOverdueAndFine()
    End Sub
    Private Sub PrintDocument1_PrintPage(sender As Object, e As Printing.
                    PrintPageEventArgs) Handles PrintDocument1.PrintPage
        Dim fontTitle As New Font("Times New Roman", 28, FontStyle.Bold,
                                        ContentAlignment.TopCenter)
        Dim fontBody As New Font("Times New Roman", 14)
        Dim invoiceNo As String = GetLastInvoiceNo()
        Dim y As Integer = 50
        e.Graphics.DrawString("Library Fine Invoice", fontTitle, Brushes.Black, 200, y)
        y += 40
        y += 40
        e.Graphics.DrawString("Invoice No: " & invoiceNo, fontBody, Brushes.Black, 50, y)
        y += 25
        e.Graphics.DrawString("Username: " & User.LoggedUserName, fontBody,
                                                Brushes.Black, 50, y)
        y += 25
        e.Graphics.DrawString("Overdue Days: " & txtOverdueDays.Text, fontBody,
                                                Brushes.Black, 50, y)
        y += 25
        e.Graphics.DrawString("Fine Amount: " & txtFine.Text, fontBody, Brushes.Black,
                                                                        50, y)
        y += 25
        e.Graphics.DrawString("Payment Mode: " & If(rbCash.Checked, "Cash", "UPI"),
                                            fontBody, Brushes.Black, 50, y)
        y += 25
        e.Graphics.DrawString("Paid Date: " & Date.Today.ToShortDateString(), fontBody,
                                                        Brushes.Black, 50, y)
        y += 40
        e.Graphics.DrawString("     Thank You!", fontBody, Brushes.Black, 250, y)
    End Sub
End Class
```

# CHAPTER 11

# System Testing

# SYSTEM TESTING

## 11.1 INTRODUCTION

System testing is a critical phase in the Software Development Life Cycle (SDLC) of the **Library Management System**. It ensures that the complete and integrated system works according to the specified functional and non-functional requirements. This phase focuses on validating the behavior of the entire application, including user modules, admin modules, database operations, and system security.

In the Library Management System, system testing verifies that features such as user registration, login authentication, book search, book issue/return, membership subscription, fine calculation, and report generation function correctly as a complete system. The primary goal of testing is to detect errors, ensure reliability, and confirm that the software meets user expectations before deployment.

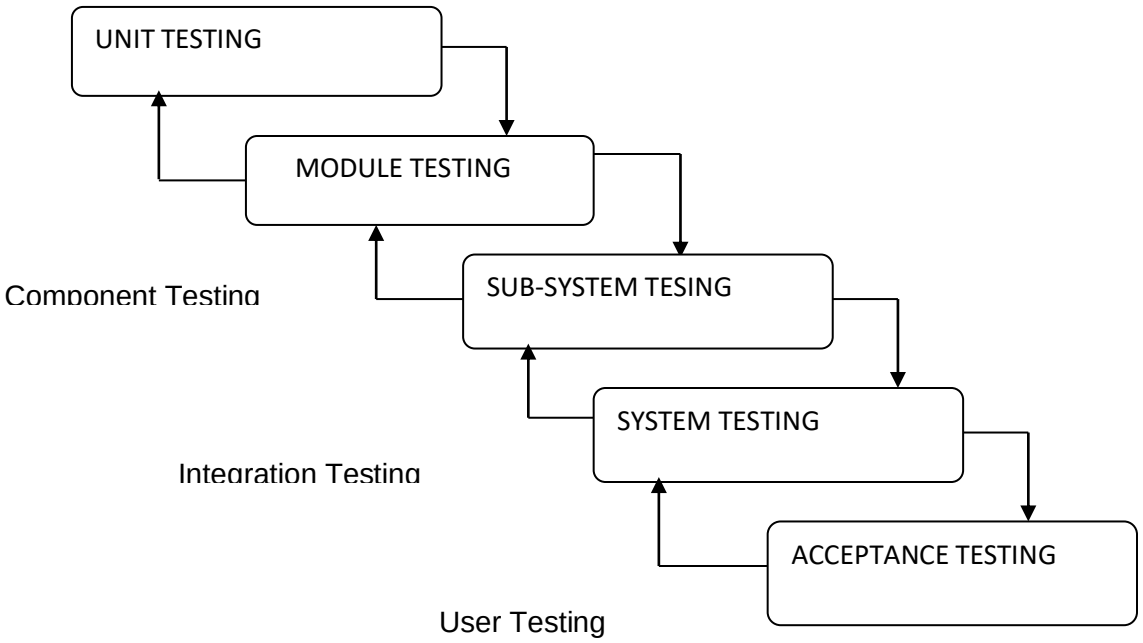
## 11.2 STRATEGIC APPROACH TO SOFTWARE TESTING

The testing strategy for the Library Management System follows a structured and systematic approach. Testing is carried out at different levels to ensure quality at each stage of development. The strategy starts from testing individual units and gradually moves toward testing the complete system.

The testing process follows the following levels:

- Unit Testing
- Module Testing
- Integration Testing
- System Testing
- Acceptance Testing

Initially, individual components such as login forms, registration forms, and database connections are tested independently. Later, these components are integrated and tested together to ensure smooth interaction between modules. Finally, system testing is performed to validate the complete Library Management System as a whole.




## 11.3 UNIT TESTING

Unit testing focuses on verifying the smallest functional units of the Library Management System. Each module is tested independently to ensure it performs its intended task correctly.

**1. WHITE BOX TESTING**

White box testing is used to test the internal logic of the system. The following conditions were verified:

- All independent paths were executed at least once
- All logical decisions were tested for both true and false conditions
- Loops were executed at their boundaries and within valid ranges
- Internal data structures were validated for correctness

This ensured proper control flow and correct execution of internal logic.

**2. BASIC PATH TESTING**

Basic path testing was performed using flow graphs and cyclomatic complexity to identify independent execution paths. This method helped in deriving test cases that ensured maximum code coverage.

The cyclomatic complexity was calculated using the formula:

- $V(G) = E - N + 2$
- $V(G) = P + 1$

**Where:**

- V(G) = Cyclomatic Complexity
- E = Number of edges
- N = Number of nodes
- P = Number of predicate nodes

### 3. CONDITIONAL TESTING

Conditional testing ensured that all conditional statements were tested for both true and false outcomes. Each decision point such as login validation, membership verification, and book availability was tested to detect logical errors.

### 4. DATA FLOW TESTING

Data flow testing was performed to ensure correct definition and use of variables throughout the program. This testing validated that data such as user details, book records, issue dates, and fine amounts were correctly stored, retrieved, and updated in the database.

### 5. LOOP TESTING

Loop testing was applied to validate loop structures used in the system, such as displaying book lists and processing transaction records.

- Loops were tested at minimum, maximum, and average limits
- Loops were skipped at least once to verify correct behavior
- Nested loops were tested starting from the innermost loop

## 11.4 PROJECT TESTING

Project testing ensures that the entire Library Management System works correctly under real-time conditions.

**1. COMPILATION TEST :** The system was successfully compiled without any errors. Compilation testing helped identify syntax errors and ensured proper integration of all modules.

**2. EXECUTION TEST :** The application was executed successfully. All modules such as user login, admin panel, book issue/return, and fine calculation executed without runtime errors.

**3. OUTPUT TEST :** All output screens such as user dashboards, book lists, issue reports, and fine details were verified. The outputs matched the expected results and were displayed correctly.

# CHAPTER 12

# Advantages & Limitation

# ADVANTAGES

The Library Management System is identified by the benefits it provides to both librarians and users. The major advantages of the system are as follows:

- The system is user-friendly and easy to operate through a graphical user interface (GUI).
- It reduces manual work and paperwork involved in maintaining library records.
- Faster access to information such as book availability, issued books, and user details.
- Improves accuracy by minimizing human errors in data entry and calculations.
- Efficient management of book issue, return, renewal, and fine calculation processes.
- Secure storage of data with proper authentication for admin and users.
- User registration and login features provide controlled access to the system.
- Membership subscription management (monthly, quarterly, yearly) is simplified.
- Saves time for librarians and improves overall productivity.
- Provides reliable and consistent output.

# LIMITATIONS

Like any software application, the Library Management System also has certain limitations that need to be considered during design and implementation.

1. **Scalability Issues**

   If the number of users, books, or transactions increases significantly, the database performance may slow down unless optimized or upgraded.

2. **Limited Multi-Platform Support**

   Since the system is primarily developed as a desktop-based application, it may not be easily accessible on mobile devices or web platforms without additional development.

3. **Dependency on Computer Systems**

   The system requires a computer and power supply to operate. In case of hardware failure or power issues, system access may be temporarily unavailable.

4. **Limited Online Features**

   Advanced features such as online book reservation, email notifications, or cloud-based access may not be available in the basic version.

5. **Security Constraints**

   Although basic authentication is implemented, advanced security mechanisms such as encryption and multi-factor authentication may not be included.

# CHAPTER 13

# CONCLUSION

# CONCLUSION

In conclusion, the **Library Management System** plays a vital role in automating and streamlining the day-to-day activities of a library. The system efficiently manages core operations such as user registration, login authentication, book management, book issue and return, membership subscription, and fine calculation. By replacing manual record-keeping with a computerized system, it significantly reduces human errors and saves time and effort for both librarians and users.

The implementation of this system improves accuracy, reliability, and efficiency in handling large volumes of data related to books, users, and transactions. It provides quick access to information such as book availability, issued books, return dates, and overdue fines. The system ensures transparency and consistency in library operations, thereby enhancing overall service quality.

The Library Management System also supports scalability and flexibility. New books, users, and membership plans can be easily added without affecting existing functionality. The system can be extended in the future to include advanced features such as online book reservations, digital library access, report generation, and notification services, making it adaptable to changing requirements.

In summary, this project successfully meets its objectives by providing a user-friendly, secure, and efficient solution for library automation. It reduces extra workload, improves data management, and ensures smooth functioning of library services. Hence, the **Library Management System** proves to be a reliable and useful application for modern library environments.

# CHAPTER 14

# BIBLIOGRAPHY

# BIBLIOGRAPHY

The following books, websites, and online resources were referred to during the analysis, design, development, and implementation of the **Library Management System** project.

## ONLINE REFERENCES


- **For .NET Framework and VB.NET Installation**
  www.support.microsoft.com
- **For VB.NET Programming and Development Support**
  www.developer.com
  www.codevedo.com/Languages/VB.net
- **For Database Design and MS-Access**
  www.codevedo.com/Database/MS_Access
  www.msdn.microsoft.com
- **For Learning VB.NET Concepts**
  http://homeandlearn.co.uk/NET/vbNet.html


## BOOKS REFERRED


- **Software Engineering**
  By Roger S. Pressman
- **MS-Access for Professionals**
  By Jain
- **Visual Basic .NET Black Book**
  By Evangelos Petroutsos


## SUMMARY

The above references provided essential guidance for understanding software engineering principles, VB.NET programming, and database management concepts required for the successful development of the Library Management System.